

ВСТУП

Сучасна розробка інтерактивного програмного забезпечення вимагає поєднання програмної логіки, введення користувачем, візуального середовища та системи обробки даних. Цей підхід особливо важливий для проектів, побудованих у середовищі Unity, де функціональність виходить за рамки використання C# та включає систему сцен, об'єктів, компонентів та налаштувань у редакторі. У цьому типі програмного забезпечення користувач може керувати персонажем, взаємодіяти з елементами середовища, отримувати предмети та використовувати інтерфейс інвентарю.

Значення цього курсу полягає в необхідності створення програмного продукту, який реалізує фундаментальні механізми взаємодії, використовуючи модель компонентів Unity та мову C#. У сучасних інтерактивних додатках недостатньо просто забезпечити переміщення персонажа або відображення інтерфейсу; це також передбачає організацію логічного зв'язку між різними підсистемами. Ці підсистеми включають керування персонажами, взаємодію з об'єктами, інвентар, обробку стану інтерфейсу та перевірку компонентів.

Керування персонажами, взаємодія з об'єктами, інвентар, обробка стану інтерфейсу та перевірка компонентів. Цей курс спрямований на проектування та розробку програмного продукту в середовищі Unity з використанням C#. Продукт забезпечить базову взаємодію користувача з 3D-середовищем, керування персонажами та системою управління інвентарем. Для досягнення цієї мети необхідно проаналізувати обсяг дослідження, виявити потенційні проблеми, дослідити аналогічні програми, обґрунтувати майбутню структуру розробки, спроектувати основні модулі системи та описати їх реалізацію.

Цей курс зосереджений на процесі розробки інтерактивного програмного продукту в середовищі Unity. Він охоплює методи проектування та реалізації компонентів керування персонажами, взаємодії з елементами сцени, організації даних персонажів та логіки управління запасами за допомогою C# та середовища Unity. Практичне значення цієї роботи полягає в можливості використання розробленого програмного продукту як навчального прикладу для побудови базової інтерактивної системи з функціональністю, розподіленою між її різними компонентами.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Особливості розробки інтерактивного програмного продукту в Unity

Розробка інтерактивного програмного продукту в середовищі Unity передбачає поєднання організації сцени, елементів середовища, введення користувачем, програмних компонентів та елементів інтерфейсу користувача. Особливістю таких програмних продуктів є те, що їхня поведінка формується не лише вихідним кодом, але й конфігурацією компонентів у сцені. Тому під час аналізу проєкту важливо враховувати не лише окремі класи C#, а й принципи їхньої взаємодії з об'єктами та компонентами Unity.

У розробленому програмному продукті механізми керування персонажем відіграють важливу роль у забезпеченні взаємодії користувача із середовищем. Користувач повинен мати можливість переміщатися простором, змінювати напрямок огляду, виконувати основні дії та взаємодіяти з об'єктами сцени. Для реалізації такої функціональності використовуються компоненти C#, які обробляють введення користувача, змінюють положення персонажа та координують взаємодію з іншими модулями системи.

Система інвентарю є окремим функціональним модулем програмного продукту. Вона забезпечує зберігання, обробку та відображення предметів, отриманих користувачем під час взаємодії з навколишнім середовищем. Така система повинна підтримувати додавання предметів, оновлення вмісту слотів, відображення кількості об'єктів та взаємодію з інтерфейсом користувача. Важливим аспектом реалізації є логічне розділення даних про предмет, його поведінки у сцені та інформації про його стан в інвентарі.

Проєкти, створені в середовищі Unity, базуються на компонентному підході до розробки програмного забезпечення. Окремі компоненти можуть відповідати за анімацію персонажа, взаємодію з предметами, роботу інвентарю або відображення інформації в інтерфейсі користувача. Такий підхід спрощує структурування проєкту, підвищує його масштабованість та полегшує подальший супровід.

1.3 Аналіз аналогів програмного забезпечення

Аналіз подібних програм дозволяє визначити рішення, які вже використовуються в порівнянних інтерактивних продуктах, та підходи, які слід враховувати під час власної розробки. У цьому проєкті доцільно зосередитися на функціональних рішеннях, пов'язаних з керуванням персонажем, взаємодією елементів та інвентарем, а не на конкретному стилі

мистецтва чи особливостях сюжету. Ці елементи більше відповідають темі програми.

У багатьох інтерактивних ігрових додатках користувач керує персонажем у просторі, взаємодіє з елементами та отримує предмети, які пізніше відображаються в інвентарі. Ключові особливості цих систем включають рух клавіатури, відображення миші, вибір елементів через взаємодію з користувачем та візуальне представлення отриманих елементів у коробках. Ці рішення зручні для користувача, оскільки вони відповідають загальній логіці взаємодії в 3D-додатках.

Під час аналізу подібних програм можна виділити кілька загальних підходів. Перший підхід передбачає появу елементів у сцені як окремих об'єктів, які переносяться до інвентарю після взаємодії. Другий підхід передбачає поділ елементів на описові дані та фактичний стан у коробках. Третій підхід пов'язаний з тим, що відкриття інвентарю змінює шаблон керування та переключає увагу користувача з руху персонажа на взаємодію з інтерфейсом користувача. Саме ці підходи слід враховувати в розроблюваному програмному продукті.

Порівняно з інтегрованими комерційними ігровими системами, розроблений програмний продукт має освітній характер і не вимагає складної системи управління предметами, розгалуженої системи обладнання або великої кількості типів взаємодії. Його завдання полягає в реалізації базової, але логічно інтегрованої системи, яка дозволяє користувачеві переміщувати та взаємодіяти з предметами, а також бачити результат цієї взаємодії в інвентарі. Такий підхід відповідає обсягу академічної роботи та дозволяє зосередитися на правильній структурі програмного продукту.

Тому аналіз подібних моделей показує, що розроблена система повинна використовувати базову модель керування персонажами, відокремлювати дані про предмети від поведінки сцени, мати централізовану підсистему інвентаризації та пов'язувати інтерфейс користувача зі станом гри. Ці рішення забезпечують чітку взаємодію з користувачем та створюють основу для майбутньої розробки програмного продукту.

2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ

2.1 UML проєктування програмного продукту

Цей проєкт охоплює розробку інтерактивного програмного продукту в середовищі Unity. Ці програмні продукти поєднують організацію сцени, елементи середовища, введення користувачем, програмні компоненти та елементи інтерфейсу користувача. Їхня унікальна характеристика полягає в тому, що поведінка системи формується не лише вихідним кодом, але й конфігурацією компонентів у сцені. Тому, під час аналізу такого проєкту важливо враховувати не лише окремі класи C#, але й те, як вони взаємодіють з елементами Unity.

У рамках розробленого програмного продукту механізми керування персонажами відіграють вирішальну роль. Користувач повинен мати можливість переміщатися по простору, змінювати свою точку зору, виконувати основні дії та взаємодіяти з елементами середовища. Для досягнення такої поведінки використовуються компоненти C#, які обробляють введення користувачем, змінюють положення персонажа та координують дії з іншими компонентами системи. Це створює основу для взаємодії, де програмний продукт реагує на дії користувача під час виконання.

Система інвентаризації є окремим компонентом програмного продукту. Вона призначена для зберігання та відображення предметів, які користувач отримує під час взаємодії зі сценою. Ця система повинна дозволяти додавати предмети, оновлювати слоти для зберігання, відображати їх кількість та взаємодіяти з інтерфейсом користувача. Важливо логічно розділити опис предмета, його поведінку в сцені та стан його інвентарю, оскільки ці частини виконують різні функції.

Проєкти Unity характеризуються компонентним підходом до розробки програмного забезпечення. Один компонент може відповідати за анімацію персонажів, інший — за взаємодію з предметом, третій — за керування елементами інвентарю, а окремий компонент — за відображення даних в інтерфейсі. Такий поділ спрощує розуміння структури проєкту та полегшує подальше розширення. Водночас він вимагає чіткого дизайну, оскільки всі модулі повинні безперебійно працювати разом.

2.2 Проєктування архітектури програмного продукту

Архітектура програмного продукту побудована відповідно до компонентної моделі Unity. У цій моделі модуль GameObject є базовою одиницею сцени, а функціональність об'єкта визначається набором плагінів.

Скрипти C# виступають як поведінкові компоненти, що відповідають за рух, взаємодію, обробку станів та зв'язок між частинами системи. Це дозволяє нам розглядати програмний продукт як сукупність модулів, кожен з яких виконує окрему функцію.

В архітектурі програмного продукту можна виділити кілька ключових модулів. Модуль контролера персонажів обробляє введення користувачем, рух, обертання камери, біг, стрибки та гравітацію. Модуль взаємодії з об'єктами визначає об'єкти, які користувач може отримати. Модуль даних об'єктів зберігає описи об'єктів окремо від їх екземплярів у сцені. Модуль інвентаризації отримує об'єкти, обробляє їх кількість та оновлює відповідні поля. Модуль інтерфейсу відображає стан інвентаризації та впливає на режим керування.

Обраний архітектурний підхід можна описати як модульний компонентно-орієнтований. Його перевагою є те, що кожна частина системи має свою власну сферу відповідальності. Компонент «Контролер символів» не потрібен для зберігання предметів, компонент «Предмети» не потрібен для самостійного керування всією системою інвентаризації, а інтерфейс користувача не повинен містити основну логіку обліку даних. Цей розділ робить програму зрозумілішою та більш придатною для майбутнього розширення.

Взаємодія між модулями відбувається через виклики функцій, посилання на компоненти та спільні стани. Наприклад, компонент «Контроль символів» може перевіряти стан інвентаризації та відповідно змінювати його поведінку. Компонент «Предмети» може передавати опис та кількість предмета менеджеру інвентаризації. У свою чергу, менеджер інвентаризації оновлює внутрішній стан та пов'язані елементи інтерфейсу користувача. Ця взаємодія забезпечує безперебійну роботу системи.

2.3 Проєктування структури зберігання даних

У програмному продукті важливо відокремити статичні дані об'єкта від змінного стану інвентаризації. Статичні дані описують сам об'єкт і можуть включати його назву, тип, опис, зображення інтерфейсу або інші властивості, якщо це визначено застосунком. Ці дані не повинні повторюватися в кожному об'єкті сцени, оскільки вони зазвичай описують ядро об'єкта. Натомість об'єкт сцени просто посилається на відповідний опис і передає його до інвентаризації під час взаємодії.

Змінний стан інвентаризації формується під час роботи програмного продукту. Цей стан містить інформацію про отримані об'єкти, їх збережену

кількість та зайняті простори. Цей стан не відповідає опису об'єкта, оскільки один і той самий тип об'єкта може мати різну кількість у різні моменти часу під час роботи системи. Тому на етапі проектування важливо відокремити опис об'єкта від поточного стану простору інвентаризації.

Підсистему інвентаризації можна представити як сукупність просторів. Кожен простір зберігає посилання на об'єкт та його кількість одиниць. Якщо об'єкт вже присутній у просторі, система може збільшити його кількість. Якщо елемент відсутній, система інвентаризації повинна шукати вільне місце. Якщо місця немає, система повинна надати відповідну відповідь, але конкретний метод обробки цієї ситуації має бути підтверджений застосунком.

Оскільки основна увага запропонованого застосунку приділяється запуску системи інвентаризації під час виконання програми, не слід вважати без подальшого підтвердження, що між сеансами відбувається повне збереження даних. Якщо ця функція існує, її слід описати окремо разом зі специфічним механізмом збереження. У поточному описі проекту доцільно пояснити внутрішню структуру станів системи інвентаризації та її зв'язок з даними про товари.

2.4 Проектування інтерфейсу користувача

Інтерфейс користувача (UI) призначений для відображення стану інвентарю та забезпечення взаємодії користувача з отриманими предметами. Його основна функція полягає в тому, щоб зробити внутрішній стан підсистеми інвентарю зрозумілим для користувача. Коли предмет додається до інвентарю, стан відповідної комірки має змінитися. Якщо предмет має кількість, це має відображатися в інтерфейсі. Якщо комірка порожня, це також має бути видно з її стану.

Під час проектування інтерфейсу важливо враховувати взаємозв'язок між інтерфейсом користувача та керуванням персонажем. Коли інвентар відкритий, користувач повинен зосередитися на взаємодії з інтерфейсом, тому повне керування персонажем може бути тимчасово обмежене. Щоб вирішити цю проблему, система забезпечує стан відкритого інвентарю, який використовується іншими компонентами. Такий підхід дозволяє уникнути ситуації, коли користувач одночасно керує камерою, рухом персонажа та елементами інтерфейсу.

Інтерфейс інвентарю повинен бути розроблений як система комірок. Кожна комірка відповідає за відображення одного предмета або порожнього стану. Коли дані інвентарю змінюються, комірка повинна оновлюватися відповідно. Це забезпечує користувачеві візуальний зворотний зв'язок після

взаємодії з предметом у сцені. Це сприяє підвищенню зрозумілості програмного продукту та забезпечує послідовну взаємодію.

У цьому курсі інтерфейс користувача слід описувати функціонально, без надмірного акценту на декоративних елементах. Основна увага має бути зосереджена на даних, що відображаються інтерфейсом користувача, на тому, як він пов'язаний з інвентаризацією та як його розміщення впливає на інші модулі. Такий підхід відповідає меті пояснювальної записки, оскільки він уточнює роль інтерфейсу користувача в загальній структурі програмного продукту.

3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ

3.1. Науково-методичне обґрунтування підготовки фрагментів курсової роботи з Unity

3.1.1 Призначення та межі постановки

Наведене вище твердження окреслює рекомендації щодо підготовки академічних розділів дослідницької роботи з розробки програмного забезпечення за допомогою Unity та C#. Її основною метою є перетворення поданих матеріалів проекту у формальний, логічно структурований та технічно точний текст. Такий підхід дозволяє нам описати не типовий проект Unity, а програмно підтримуваний додаток, сцени, шаблони, знімки екрана, ресурси чи інші елементи. Це особливо важливо для курсової роботи, оскільки академічний виклад повинен спиратися на точність, об'єктивність, формальність і перевірюваність тверджень [1].

У виробництві ретельно розрізняються факти, які можна зробити з матеріалів проекту, та дані, що потребують подальшого підтвердження. Виходячи з цієї логіки, заборона на створення нових функцій, структур, результатів тестування, технічних специфікацій та джерел систематично обґрунтовується. Це має вирішальне значення для проекту Unity, оскільки одну й ту саму зовнішню поведінку в ігровій сцені можна реалізувати по-різному. Наприклад, взаємодія з об'єктом може залежати від компонента `MonoBehaviour`, конфігурації `ScriptableObject`, попередньо створеної структури, зовнішнього пакета або налаштувань сцени.

Однак, виробнича команда повинна чітко визначити матеріали, які є основними доказами для технічного опису. Ці матеріали включають код C#, сцени, попередньо створені моделі, скріншоти інспектора та ієрархії, файли `Packages/manifest.json`, файли `.asmdef`, звіти про тестування, журнали та офіційні сторінки для використаних ресурсів. Ця документація дозволяє нам визначити, що було реалізовано, як це працює та з якими компонентами системи це взаємодіє. Документацію `Public Unity`, ресурси `Microsoft` та академічні джерела слід використовувати як підтверджуючі докази для пояснення перевірених рішень, а не як основу для приписування відсутньої функціональності проекту.

Джерело даних	Що можна описувати без ризику вигадування	Чого не слід стверджувати без додаткового доказу
C#-код	призначення класів, методів, залежностей, перевірок і викликів API Unity	фактичну продуктивність, UX-ефект або результати тестування
Сцени, префаби, Inspector	склад об'єктів, компоненти, серіалізовані поля, посилання між об'єктами	динамічну поведінку, яка не підтверджена кодом

Тестові звіти й логи	сценарії перевірки, фактичні результати, знайдені помилки	загальну стабільність системи без зафіксованих перевірок
Asset Store, README, EULA	назву пакета, ліцензію, версію, розмір, сумісність	спосіб інтеграції пакета в конкретний проєкт без артефактів

Наведена нижче таблиця визначає межі прийнятного опису та зменшує ризик підміни аналізу припущеннями. Рекомендується використовувати її як методологічну основу під час написання розділів про архітектуру системи, інтерфейс, взаємодію, тестування та ресурси. У цьому випадку кожне технічне твердження матиме чітке підґрунтя в конкретному матеріалі проєкту.

3.1.2 Технічна база Unity та C#

Для коректного опису Unity-проєкту постановка має спиратися на компонентну модель рушія. У Unity сцена використовується як окремий простір організації ігрового середовища, меню, об'єктів і декоративних елементів [2]. GameObject є базовою сутністю сцени та контейнером для функціональних компонентів, які визначають зовнішній вигляд і поведінку об'єкта [3]. Тому, описуючи процес реалізації, слід пояснювати не лише наявність об'єкта в сцені, але й компоненти, що складають його функціональну роль у системі.

Скрипти MonoBehaviour у Unity виступають компонентами, які можна прикріплювати до GameObject для реалізації поведінки. [Це означає, що опис класу в курсовій роботі повинен враховувати його зв'язок зі сценою, інспектором, серіалізованими полями та іншими компонентами.] Якщо в коді використано атрибут SerializeField, його слід пояснювати як механізм серіалізації приватних полів, що дає можливість налаштовувати значення через Inspector без відкриття доступу до поля як public. Такий опис пов'язує програмну інкапсуляцію з практичним налаштуванням поведінки об'єктів у редакторі Unity.

Скрипти MonoBehaviour в Unity діють як компоненти, які можна додавати до GameObject для реалізації певних моделей поведінки. Це означає, що опис класу в присвоєнні повинен враховувати його зв'язок зі сценою, Інспектором, серіалізованими полями та іншими компонентами. Якщо код використовує властивість SerializeField, це слід пояснити як механізм серіалізації приватних полів, що дозволяє встановлювати значення через Інспектор, не роблячи поле публічним. Цей опис усуває розрив між інкапсуляцією коду та практичною конфігурацією поведінки об'єкта в редакторі Unity.

Окрему роль у технічному описі можуть виконувати ScriptableObject-активи. Unity визначає ScriptableObject як тип, який дає змогу зберігати дані незалежно від GameObject і звертатися до них за посиланням, зменшуючи дублювання даних у префабах [4]. Однак, ця інтерпретація дійсна лише тоді, коли вхідні дані фактично надають ScriptableObject або код, який з ним

працює. Якщо відповідний артефакт відсутній, у курсовій роботі не слід стверджувати, що проєкт використовує конфігураційні активи такого типу.

Для C#-частини постановка повинна вимагати аналізу класів, методів, інтерфейсів, просторів імен і залежностей. Класи є основним засобом опису користувацьких типів у C#, а інтерфейси використовуються для формалізації контрактів поведінки між частинами програми [5]. У дослідницькій роботі це дозволяє пояснити не лише окремих фрагмент коду, але й архітектурну роль компонента в системі в цілому. Такий підхід особливо корисний під час опису модулів інвентарю, взаємодії з об'єктами, керування інтерфейсом або збереження даних.

До обов'язкових матеріалів для якісного опису Unity-проєкту доцільно віднести такі артефакти:

- Packages/manifest.json або скріншот Package Manager для фіксації залежностей і версій пакетів;
- .asmdef-файли, структуру тек і простори імен для опису модульності;
- скріншоти Hierarchy, Inspector, ключових сцен і префабів для підтвердження композиції системи;
- C#-код для аналізу класів, методів, викликів і перевірок;
- тестові таблиці, логи або звіти Unity Test Framework для підтвердження результатів перевірки.

Цей перелік забезпечує трасованість між фактичними матеріалами проєкту та підсумковим текстом курсової роботи. Про цей вид роботи доступно багато інформації. Дізнайтеся більше про програмне забезпечення Unity.

3.1.3 Суперечності постановки та спосіб їх усунення

Основна внутрішня суперечність у початковому твердженні полягає в одночасній забороні створення нових функціональних можливостей та можливості автоматичного вставки фрагмента коду, якщо фактичний код не надано. З академічної точки зору, така практика є проблематичною, оскільки академічна робота повинна описувати конкретний проєкт, а не бути відповідним прикладом за певних умов. Фрагмент коду слід використовувати лише як ілюстративний приклад, а не як доказ фактичної реалізації. Тому важливо чітко зазначити в твердженні, що відсутній код не може бути замінений стандартним рішенням без визнання його ілюстративного характеру.

Якщо як приклад використовується скрипт PlayerRaycast, його слід описати з великою технічною точністю та ретельністю. Цей скрипт може реалізувати взаємодію з об'єктами через промінь, що випромінюється з позиції камери, перевірити наявність зіткнень за допомогою Physics.Raycast та спробувати отримати потрібний компонент за допомогою TryGetComponent. Мета цього скрипта — ідентифікувати локальний об'єкт для гравця та передати керування відповідному компоненту взаємодії. Водночас використання Input.GetKeyDown необхідно описувати як роботу з legacy Input

Manager, оскільки Unity прямо вказує, що цей API не рекомендовано застосовувати в нових проєктах [6].

Безпечний процес підготовки тексту доцільно подати як послідовний конвеєр:

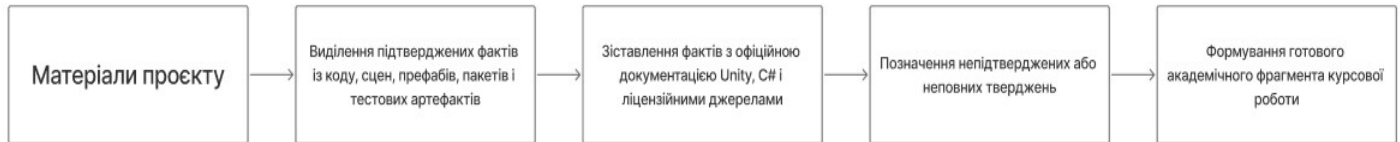


Рисунок 1 – Конвеєр верифікації фактів перед підготовкою академічного тексту

Наведена вище діаграма ілюструє, що процес написання не повинен починатися з упереджених припущень щодо типової архітектури проєкту Unity, а радше з аналізу доступних матеріалів. Кожен наступний етап уточнює, які дані можна вважати остаточними, а які потребують додаткових джерел або матеріалів. Така послідовність підвищує достовірність тексту та підтримує узгодженість між фактичним застосуванням та його описом у пояснювальній записці.

На практиці це означає три правила. По-перше, опис фактичного застосування дозволяється лише за наявності доказів, що підтверджують цей опис. По-друге, тестовий код не повинен бути представлений як частина проєкту без безпосереднього тестування. По-третє, розділи, що стосуються тестування, продуктивності, стабільності або збереження даних, не повинні обмежуватися загальними висновками без журналів тестування, таблиць або звітів про тестування.

3.1.4 Опис архітектури, інтерфейсу, даних і тестування

Архітектурний опис курсової роботи повинен виходити з реальних меж модулів. Стандарт ISO/IEC/IEEE 42010 пов'язує архітектурний опис із зацікавленими сторонами, їхніми потребами, поданнями системи та відповідними архітектурними рішеннями [7]. У контексті Unity цими представленнями можуть бути сцени, попередньо створені шаблони, програмні модулі, ресурси ScriptableObject, функції пакета та канали даних між компонентами. Саме тому опис архітектури не повинен обмежуватися переліком класів, а має пояснювати їхню роль у системі.

Якщо в проєкті використано Assembly Definition файли, їх потрібно описувати як засіб організації коду в окремі збірки. Unity вказує, що .asmdef-файли застосовуються для створення або редагування визначень збірок, а практики організації проєкту рекомендують використовувати простори імен і впорядковувати структуру коду для зменшення конфліктів зі сторонніми активами [8]. Тому в академічному тексті важливо пояснювати, як модуль пов'язаний не лише з класами, а й зі збіркою, простором імен, сценою або префабом. Це забезпечує архітектурний контекст і робить опис системи змістовнішим.

Опис інтерфейсу користувача також має будуватися від доказів до висновків. Якщо матеріали підтверджують використання UI Toolkit, його можна описувати як набір засобів Unity для створення інтерфейсів користувача [9]. Якщо надається лише зображення списку без значка, подій або налаштувань об'єкта, то структура екрана, призначення кнопок та логіка навігації можуть бути описані на рівні зовнішнього скрипта. У такому випадку не слід вигадувати конкретну реалізацію обробників подій або внутрішню систему переходів між екранами.

Розділи про збереження даних потребують особливої точності. Якщо в коді використано PlayerPrefs, можна зазначити, що цей механізм зберігає значення між сесіями та підтримує рядкові, цілі й дійсні значення. Водночас Unity прямо вказує, що PlayerPrefs зберігає дані локально без шифрування, тому його не слід використовувати для чутливої інформації [10]. Якщо в матеріалах проєкту немає коду збереження, то твердження щодо системи збереження повинні залишатися непідтвердженими.

Опис тестування має спиратися тільки на надані тестові матеріали. Unity Test Framework підтримує перевірки в Edit Mode і Play Mode, що дає змогу тестувати як редакторну логіку, так і поведінку в режимі виконання [11]. Однак сама по собі наявність фреймворку не доводить, що тести були проведені або що в конкретному проєкті були отримані конкретні результати. Тому без таблиць перевірки, звітів або логів можна описувати лише запланований підхід до тестування, а не фактичну стабільність системи.

Категорія твердження	Мінімальний доказ	Допустиме формулювання
Архітектурна роль класу	код, namespace, .asmdef, Inspector	клас виконує функцію керування або взаємодії з визначеними компонентами
UI-логіка	скріншот, сценовий об'єкт, код подій	інтерфейс містить елементи керування та забезпечує навігацію між станами
Збереження даних	код PlayerPrefs, JsonUtility або файлової системи	дані зберігаються за допомогою підтвердженого механізму
Обробка помилок	перевірки null, Debug.LogError, винятки	реалізовано перевірку валідності посилань або логування помилок
Результати тестування	тестовий звіт, лог, таблиця перевірки	під час тестування встановлено конкретний результат

Ця таблиця допомагає зберегти достовірність технічного опису. Вона також дозволяє відокремити перевірені аналізи від припущень, які потребують додаткових даних.

3.1.5 Робота з асетами та ліцензійними обмеженнями

Під час опису сторонніх асетів потрібно відрізнити технічні характеристики пакета від факту його реального використання в проєкті. Asset Store Terms of Service and EULA визначає правові умови використання ресурсів Unity Asset Store, а довідкові матеріали Unity пояснюють типи ліцензій, зокрема Extension Asset, Single Entity і Multi-Entity [12]. Тому пошук має окремо включати джерело програмного забезпечення, тип ліцензії, версію, розмір та сумісність, а також те, як воно інтегровано в певну сцену чи систему. Просте завантаження або володіння ресурсом не є доказом того, що його було змінено, покращено або програмно пов'язано з іншими компонентами.

Офіційна сторінка Abandoned Asylum в Unity Asset Store вказує, що пакет поширюється безкоштовно, має Standard Unity Asset Store EULA, тип ліцензії Extension Asset, розмір 1.8 GB, версію 1.4, дату останнього релізу 17 липня 2020 року та original Unity version 2019.4.1 [13]. Цю інформацію можна використовувати в академічних дослідженнях як остаточний опис пакету. Однак кількість кімнат, моделей, матеріалів, наявність повноцінного театру або фактичне використання конкретних предметів у студентському проєкті має бути задокументовано з окремих джерел.

Офіційна сторінка 3D Low Poly Magical Forest містить дані про розмір 5.7 MB, версію 1.1.0, дату останнього релізу 13 травня 2026 року та original Unity version 2022.3.26 [14]. Ці критерії можна вважати довідковою інформацією про пакет, якщо вони вже використовувалися раніше в проєкті. Однак, конкретний перелік дерев, грибів, кристалів або декоративних структур, або як їх використовувати в сценах, не слід описувати без скріншотів, імпортованих файлів або додаткових вихідних матеріалів.

Free Wood Door Pack в Unity Asset Store подано як безкоштовний пакет зі Standard Unity Asset Store EULA, типом ліцензії Extension Asset, розміром 99.3 MB, версією 1.0, датою релізу 22 квітня 2024 року та original Unity version 2021.3.32 [15]. У дослідженнях ці дані можуть бути використані для ліцензування та технічного опису для постачальника. Однак заяви щодо кількості моделей дверей, наявності системи відкривання або будь-яких конкретних змін у дизайні повинні бути підтверджені власними матеріалами.

POLY - Lite Survival Collection на сторінці Unity Asset Store зазначено як безкоштовний пакет зі Standard Unity Asset Store EULA, типом ліцензії Extension Asset, розміром 3.3 MB, версією 1.0, датою релізу 6 травня 2024 року та original Unity version 2020.3.0 [16]. Цих даних достатньо для базового опису пакета, але недостатньо, щоб зробити висновки про його повну структуру чи цілісність. Якщо потрібна підтримка конкретних моделей, зброї, текстур, полігонів або шляхів відображення, необхідно надати додаткові докази.

З ліцензійного погляду важливо не лише назвати пакет, а й пояснити обмеження його використання. Unity Support зазначає, що Extension Assets потребують місця для кожного користувача, який має доступ до сирих файлів, тоді як Single Entity і Multi-Entity мають іншу логіку застосування [17]. Отже,

документування ліцензування в дослідницькій роботі — це не просто формальність; це частина підтвердження законного використання ресурсів третіх сторін. Якщо в тексті стверджується, що ресурс був змінений, адаптований або перемальований, самі матеріали проекту повинні підтверджувати це твердження.

3.1.6 Рекомендована редакція постановки

Зберігаючи академічну об'єктивність, технічну точність та простежуваність структури проекту, доцільно розробляти маніфест проекту як посібник з опису перевіреної реалізації Unity. Маніфест повинен чітко відокремлювати деталі проекту від загальних технічних пояснень та демонстраційних прикладів. Особливо важливо уникати невідповідностей між простим у виробництві та автоматичним додаванням модульного коду. Це дозволить вам написати дослідницьку роботу, яка відповідає фактичній структурі проекту, а не замінювати аналіз модульними рішеннями.

Рекомендована презентація повинна містити назву розділу, фрагменти коду, знімки екрана, сцени, прототипи, використані ресурси, дані пакетів та тестові матеріали. На основі цієї документації технічний автор повинен описати призначення компонентів, як вони функціонують, як вони взаємодіють з іншими частинами системи та їхню роль у загальній архітектурі. Якщо матеріали не підтримують певний аспект, його не слід винаходити або замінювати загальним описом. Такий підхід відповідає вимогам академічної доброчесності та технічної перевіреності.

На практиці, ця версія маніфесту найкраще підходить для написання послідовних розділів пояснювальної записки. Вона дозволяє описувати проект Unity шляхом пов'язування сцен, ігрових елементів, компонентів, коду C#, пакетів, системи інтерфейсу користувача, системи введення, ресурсів та тестової документації. Завдяки цьому навчальна програма зберігає свій науковий стиль, одночасно зберігаючи точність технічного опису. Такий формат маніфесту мінімізує ризик помилкових тверджень та забезпечує узгодженість між реалізацією програмного продукту та його академічною презентацією.

3.2 Реалізація підсистем керування персонажем та інтеграції ресурсів

3.2.1 Загальна характеристика програмної реалізації

Надані матеріали містять достатньо доказів, щоб підтвердити, що програмний продукт реалізовано в середовищі Unity з використанням C# та компонентної моделі для побудови сцени. У цій моделі GameObject є базовою сутністю, а його поведінка визначається набором компонентів, що визначають функціональність об'єкта в сцені. Компонентний підхід дозволяє розділити візуальні, фізичні та програмні властивості об'єкта, спрощуючи налаштування сцени та підтримку проекту. Для ілюстрації процесу впровадження варто

зазначити, що логіка програми в Unity зазвичай додається за допомогою скриптів C#, які діють як компоненти для об'єктів сцени.

PleyrControl — це ключовий компонент логіки програми, що відповідає за керування рухом персонажа, обертанням камери, стрибками та контактом із землею. Цей компонент діє як центральний блок керування для гравця, поєднуючи введення користувача з рухом елементів у ігровому середовищі. Архітектурно він спирається на CharacterController, стандартний інструмент Unity для керування персонажем без використання жорсткого тіла. Цей підхід припускає, що рух не контролюється автоматично фізичним движком, а явно активується викликом функції Move().

Код також реєструє зовнішню залежність від InventoryManager.IsInventoryOpen, що вказує на взаємодію контролера персонажа з модулем інвентарю. Ця залежність є важливою для розрізнення двох режимів роботи: активного керування персонажем та стану, коли увага користувача зосереджена на інтерфейсі інвентарю. Якщо інвентар відкритий, компонент не виконує обертання або горизонтального руху, а продовжує обробляти вертикальну швидкість. Така конструкція дозволяє підтримувати основну фізичну поведінку персонажа, навіть коли основний контроль тимчасово обмежений.

Елемент	Тип	Призначення в системі
PleyrControl	MonoBehaviour	Координує логіку переміщення, повороту камери та стрибка
_characterController	CharacterController	Виконує переміщення персонажа з урахуванням зіткнень
_cameraTransform	Transform	Забезпечує вертикальний поворот камери
_checkGroundTransform	Transform	Задає точку перевірки контакту із землею
_groundMask	LayerMask	Визначає шари, які вважаються поверхнею
InventoryManager.IsInventoryOpen	зовнішня статична залежність	Блокує частину рухової логіки під час відкриття інвентаря

У таблиці показано основні залежності контролера персонажа. Кожен елемент виконує окрему функцію, але в компоненті `PleyrControl` вони об'єднані в один цикл для обробки поведінки гравця. Це дозволяє контролеру реагувати на вхідні дані, змінювати положення персонажа, керувати камерою та одночасно контролювати стан інтерфейсу користувача.

Діаграма ілюструє послідовність виконання основної логіки в одному кадрі. Центральною точкою керування є функція `Update()`, яка визначає, чи повинен персонаж повністю реагувати на введення користувача, чи просто рухатися вертикально. Такий поділ дозволяє координувати роботу модуля руху та модуля інвентаризації, уникаючи конфліктів між режимами рендерингу сцени та взаємодії з інтерфейсом.

3.2.2 Реалізація модуля керування персонажем

У наведеному лістингу метод `Start()` переводить курсор у заблокований стан і робить його невидимим. Це рішення відповідає режиму керування від першої особи або режиму вільного огляду, коли рух миші використовується не для переміщення курсора по екрану, а для повороту камери. Така поведінка потрібна для того, щоб користувач сприймав мишу як інструмент керування оглядом у тривимірному просторі. Водночас цей механізм повинен узгоджуватися з інтерфейсом інвентаря, оскільки для взаємодії з UI курсор може потребувати іншого стану.

Метод `Update()` виступає центральною точкою синхронізації всієї поведінки компонента. У кожному кадрі він перевіряє стан `InventoryManager.IsInventoryOpen` і визначає, який набір дій має бути виконаний. Якщо інвентар відкрито, виконання методів `Rotate()` і `Move()` припиняється, але метод `Velocity()` продовжує працювати для підтримання Звертикального стану персонажа. Якщо інвентар не відкрито, компонент послідовно обробляє поворот, горизонтальне переміщення та вертикальну швидкість.

Метод `Rotate()` реалізує окремий контур керування оглядом і використовує значення віртуальних осей `Mouse X` та `Mouse Y`. Отримані значення множаться на `_sensitivity` і `Time.deltaTime`, що дозволяє регулювати чутливість і зменшувати залежність руху від частоти кадрів. Вертикальний кут накопичується у змінній `rotationX`, після чого обмежується функцією `Mathf.Clamp()` у діапазоні від `-90f` до `90f`. Це запобігає надмірному обертанню камери вгору або вниз і забезпечує стабільне сприйняття простору користувачем.

Метод `Move()` відповідає за горизонтальне переміщення персонажа. Він перевіряє натискання клавіш `W`, `A`, `S` і `D`, після чого формує напрям руху на основі локальних векторів `transform.forward` та `transform.right`. Завдяки цьому персонаж рухається не у глобальних координатах сцени, а відносно власного повороту, що є необхідним для керування від першої особи. Нормалізація вектора `move` усуває небажане збільшення швидкості під час діагонального руху, коли одночасно натискаються дві клавіші.

Перемикання між звичайним рухом і бігом реалізовано через перевірку клавіші LeftShift. Якщо клавіша натиснута і вектор руху не дорівнює нулю, застосовується швидкість `_speedRun`, інакше використовується базова швидкість `_speed`. Таке рішення забезпечує два режими переміщення без створення окремого станового автомата. Переміщення передається в `CharacterController.Move()` як абсолютний вектор зсуву за кадр, тому значення додатково масштабується через `Time.deltaTime`.

Метод `Velocity()` відповідає за перевірку контакту з поверхнею, обробку стрибка та застосування гравітації. Контакт із землею перевіряється через `Physics.CheckSphere()` у точці `_checkGroundTransform.position` з урахуванням радіуса `_checkRadiusSphere` і маски шарів `_groundMask`. `LayerMask` у цьому випадку використовується для того, щоб система розпізнавала як землю лише ті об'єкти, які належать до визначених шарів. Якщо персонаж перебуває на землі й вертикальна швидкість від'ємна, значення `velocity.y` встановлюється на `-2f`, що допомагає утримувати персонажа в контакті з поверхнею.

Стрибок реалізовано через перевірку `Input.GetKeyDown(KeyCode.Space)` за умови, що персонаж перебуває на землі. Значення вертикальної швидкості обчислюється через `Mathf.Sqrt(_jumpHeight - 2f * _gravity)`, що дозволяє пов'язати висоту стрибка з параметром гравітації. Після цього до вертикальної швидкості кожного кадру додається гравітаційне прискорення, а отриманий вектор передається в `CharacterController.Move()`. Таким чином, вертикальна динаміка винесена в окремий метод і логічно відокремлена від горизонтального руху.

3.2.3 Модульність, принципи ООП та архітектурна роль компонентів

З позиції об'єктно-орієнтованого програмування клас `PleyrControl` демонструє принцип інкапсуляції. Більшість полів оголошено як `private`, але завдяки атрибуту `[SerializeField]` вони залишаються доступними для налаштування в `Inspector`. Таке поєднання дозволяє поєднати конфіденційність внутрішнього стану класу з практичною можливістю налаштування компонента в редакторі Unity. Такий підхід є доцільним для студентського Unity-проекту, оскільки параметри швидкості, чутливості, гравітації та стрибка можна змінювати без редагування коду.

Поділ логіки на методи `Rotate()`, `Move()` і `Velocity()` покращує локальну зрозумілість реалізації. Кожен метод відповідає за окремий аспект поведінки персонажа: огляд, горизонтальне переміщення або вертикальну динаміку. Це спрощує читання коду, оскільки центральний метод `update()` не містить безпосередньо всієї логіки, а лише координує виклик спеціалізованих процедур. Водночас у межах одного класу залишаються обробка вводу, робота з камерою, перевірка землі та взаємодія з інвентарем, тому модуль не можна вважати повністю декомпонованим.

Архітектурно компонент має дві важливі зовнішні залежності. Перша залежність пов'язана із системою вводу Unity, оскільки клас напямую використовує `Input.GetAxis()`, `Input.GetKey()` і `Input.GetKeyDown()`. Друга

залежність пов'язана з `InventoryManager.IsInventoryOpen`, який впливає на доступність руху та повороту. Це рішення просте та зрозуміле, але воно збільшує взаємозалежність між контролером персонажів та зовнішнім станом інвентарю.

До сильних сторін поточної побудови можна віднести такі характеристики:

- використання закритих серіалізованих полів для налаштування параметрів через `Inspector`;
- поділ логіки на методи, які відповідають за окремі частини поведінки;
- нормалізацію вектора руху для усунення збільшення швидкості по діагоналі;
- перевірку контакту із землею через окрему точку й маску шарів;
- урахування стану інвентаря під час обробки керування.

Наведені ознаки свідчать про те, що компонент має зрозумілу внутрішню структуру та виконує визначену роль у системі. Поєднує введення користувача, рух персонажа, обертання камери та обмеження, пов'язані з відкриттям інтерфейсу інвентарю. Разом із тим для підвищення модульності в майбутньому можна винести обробку вводу, керування камерою або перевірку стану інвентаря в окремі компоненти.

До обмежень реалізації належить пряме використання класу `Input`, що прив'язує компонент до класичного `Input Manager`. Оскільки `Input Manager` класифікується як застаріле рішення в сучасній документації Unity, текст курсу повинен точно описувати його як механізм, що використовується в проекті, а не як універсально рекомендований підхід для нових проектів. Також варто звернути увагу на назву `PleyrControl`, яка має ознаки орфографічної помилки. Для остаточної редакції проекту доцільно узгодити назви класів із єдиним стилем іменування, наприклад `PlayerControl`.

3.2.4 Користувацький інтерфейс, збереження даних та обробка помилок

У наданому коді не представлено повну реалізацію користувацького інтерфейсу, однак звернення до `InventoryManager.IsInventoryOpen` свідчить про зв'язок модуля керування персонажем із системою інвентаря. Це означає, що інтерфейс інвентарю впливає не лише на те, як предмети відображаються на екрані, але й на поведінку персонажа в сцені. Коли інвентар відкрито, рухова логіка частково блокується, що запобігає одночасному керуванню персонажем і взаємодії з інтерфейсом. Така взаємодія між UI-модулем і контролером є важливим елементом загальної логіки програмного продукту.

Оскільки в методі `Start()` курсор блокується й приховується, система інвентаря повинна мати механізм зміни стану курсора або альтернативний спосіб навігації. Якщо користувачеві потрібно взаємодіяти з кнопками, полями або предметами інвентарю, вказівник має бути доступним під час відкриття інтерфейсу користувача або керування має бути забезпечене іншим способом. У наданому фрагменті така логіка не показана, тому її не можна

описувати як реалізовану без додаткових матеріалів. Коректним є твердження, що поточний контролер уже враховує стан інвентаря, але повна UI-логіка потребує підтвердження відповідним кодом або скріншотами.

Механізми збереження даних у наданих матеріалах не продемонстровані. Таким чином, неможливо стверджувати, що проєкт дійсно виконує збереження налаштувань, прогресу, інвентарю чи стану сцени. Якщо в подальших матеріалах буде показано використання `PlayerPrefs`, його можна буде описати як засіб збереження простих значень між сесіями. Якщо буде надано код із `JsonUtility` або файловою системою, тоді доцільно описати серіалізацію складніших структур даних.

У частині обробки помилок поточний фрагмент має обмеження, оскільки не містить явних перевірок серіалізованих посилань перед їх використанням. Правильна робота компонента залежить від того, чи були встановлені посилання на `CharacterController`, `Camera`, `Ground Checkpoint` та `Layer Mask` в Інспекторі. Якщо будь-яке з цих посилань не буде налаштоване, під час виконання можуть виникнути помилки. Для підвищення надійності доцільно додати перевірку обов'язкових залежностей і логування некоректної конфігурації.

До аспектів, які потребують уточнення в остаточній версії пояснювальної записки, належать:

- спосіб реалізації інтерфейсу інвентаря;
- механізм зміни стану курсора під час відкриття UI;
- наявність або відсутність системи збереження даних;
- спосіб валідації посилань, налаштованих через Inspector;
- фактичні сценарії тестування модуля керування персонажем.

Ці уточнення не змінюють підтвердженої логіки класу `PleyrControl`, але впливають на повноту опису програмного продукту. Це необхідно для того, щоб розділ курсу не лише відображав окремий текст, а й відображав взаємодію керування персонажами з інтерфейсом, сценою, ресурсами та даними.

3.2.6 Стабільність роботи, тестування та оцінка реалізації

З точки зору стабільності виконання позитивним є використання `Time.deltaTime` у розрахунках повороту, горизонтального руху та вертикальної швидкості. Це дозволяє зменшити залежність поведінки персонажа від частоти кадрів і зробити керування більш передбачуваним. Додатковою перевагою є перевірка землі через `Physics.CheckSphere()` з використанням `LayerMask`, оскільки вона обмежує перелік об'єктів, які можуть вважатися поверхнею. Такий підхід підвищує контрольованість взаємодії персонажа зі сценою.

Разом із тим у поточній реалізації метод `CharacterController.Move()` викликається двічі за кадр: один раз у `Move()` для горизонтального руху і другий раз у `Velocity()` для вертикальної швидкості. З функціональної точки зору це дозволяє розділити логіку, але з архітектурної точки зору доцільніше об'єднати горизонтальний та вертикальний вектори в єдиний кінцевий вектор руху. Така зміна зробила б рух персонажа більш цілісним і полегшила б

подальше тестування. При цьому сама логіка ходьби, бігу, стрибка й гравітації могла б залишитися незмінною.

Інформація про фактично виконані тести в наданих матеріалах відсутня, тому не можна стверджувати про підтверджену стабільність, продуктивність або повну відсутність дефектів. Тільки ті випробування, які можна виконати або які мають бути підтверджені елементами випробувань, вважаються валідними. Для Unity-проєкту доцільно розглядати як ручні сценарії перевірки, так і автоматизовані тести через Unity Test Framework. Однак результати таких тестів мають бути додані окремо у вигляді таблиці, скріншота Test Runner або журналу виконання.

До рекомендованих сценаріїв перевірки модуля належать:

1. Перевірка переміщення вперед, назад, ліворуч і праворуч.
2. Перевірка нормалізації діагонального руху.
3. Перевірка перемикання між ходьбою та бігом при натисканні LeftShift.
4. Перевірка стрибка лише за умови контакту із землею.
5. Перевірка блокування повороту й горизонтального руху під час відкриття інвентаря.
6. Перевірка правильності призначення серіалізованих посилань у Inspector.

Наведені сценарії охоплюють основну функціональність компонента `PleyrControl` і можуть бути використані як основа для таблиці тестування. Це дозволяє протестувати не лише окремі стилі, але й загальну поведінку персонажа в реальних сценаріях. Після виконання перевірок у курсовій роботі потрібно подати фактичні результати, а не лише очікувану поведінку системи.

Отже, наданий фрагмент реалізації підтверджує наявність базового модуля керування персонажем у Unity з використанням `C#`, `CharacterController`, серіалізованих полів і компонентного підходу. Найбільш документованим елементом є сам контролер символів, тоді як інтерфейс інвентаризації, збереження даних, повна обробка помилок та результати тестування потребують додаткових матеріалів. У підсумку цей розділ може бути використаний як технічна основа пояснювальної записки, але перед остаточним поданням його бажано доповнити скріншотами сцени, Inspector, ієрархії об'єктів, використаних асетів і тестових результатів.

3.3 Програмна реалізація та інтеграція ресурсів у Unity

3.3.1 Архітектурна організація програмного продукту

Програмний продукт реалізується в середовищі Unity із використанням мови `C#`, що визначає компонентний характер побудови застосунку. У Unity базовою сутністю сцени є `GameObject`, а його функціональність формується через компоненти, які додаються до об'єкта та визначають його поведінку, стан і взаємодію з іншими елементами сцени [4]. Такий підхід дозволяє розділити візуальне представлення, дані та логіку програми, не створюючи суворої ієрархії між усіма об'єктами системи. На рівні курсової роботи це дає

підстави розглядати сцену не як набір ізольованих моделей, а як систему взаємопов'язаних компонентів.

Сценарії, які реалізують прикладну поведінку, створюються як класи C#, що наслідують `MonoBehaviour` і прикріплюються до `GameObject` у вигляді компонентів [5]. Це означає, що логіка програми для проєкту Unity реалізується не лише у вихідному коді, але й шляхом прив'язки цього коду до певних об'єктів у сцені. У такій моделі клас описує поведінку, `GameObject` надає контекст існування, а `Inspector` забезпечує налаштування параметрів компонента. Завдяки цьому архітектура проєкту поєднує об'єктно-орієнтовані принципи C# із редакторною моделлю Unity.

З наданих матеріалів можна виокремити кілька логічних шарів реалізації. Перший шар утворюють сценові компоненти взаємодії, які розміщуються на `GameObject` і реагують на дії користувача або події ігрового середовища. Другий шар представлений об'єктами даних типу `ScriptableObject`, які використовуються для винесення описів предметів за межі конкретних екземплярів у сцені [6]. Третій рівень складається з управлінських служб, зокрема менеджера запасів, який координує зміни в стані запасів та приймає замовлення від інтерактивних компонентів.

Основні модулі, які простежуються з наданих матеріалів, доцільно подати так:

- модуль сценової взаємодії реалізує поведінку конкретного об'єкта в ігровому просторі, зокрема підбір предмета;
- модуль предметних даних містить опис предмета у форматі `ScriptableObject` і відокремлює довідкову інформацію від сценового екземпляра;
- модуль керування інвентарем виконує роль координатора, який приймає запити на додавання предметів;
- модуль ресурсного наповнення сцени охоплює імпортовані асети, `prefab`-об'єкти, матеріали та налаштування візуального середовища.

Наведена вище структура демонструє модульні організаційні характеристики. Логіка вибору, опису та розрахунку вартості запасів не зосереджена в одній категорії, а розподілена між різними елементами системи. Однак надані матеріали не містять повного переліку категорій, внутрішніх контрактів та методів взаємодії між усіма підрозділами. Тому ці аспекти потребують уточнення за допомогою додаткових фрагментів коду, скріншотів з вікна огляду, структури сцени або опису менеджера запасів.

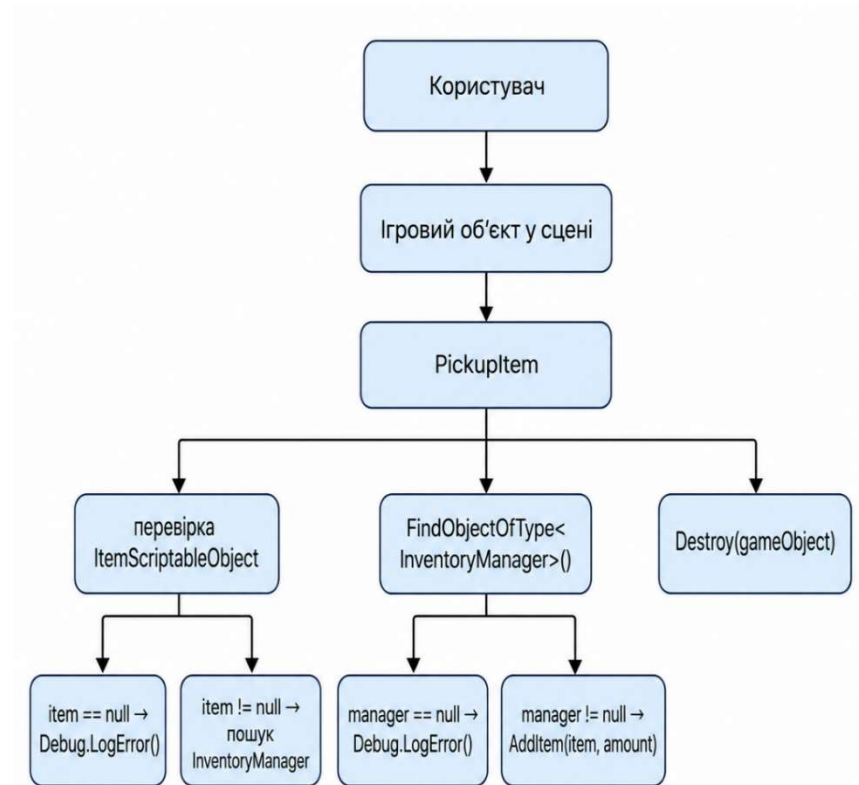


Рисунок 2 – Логічна взаємодія модулів під час підбору предмета

Схема відображає послідовність виконання операції підбору предмета. Це показує, що фазовий елемент безпосередньо не змінює інвентар, а радше делегує відповідальність за зміну статусу окремому менеджеру. Така взаємодія зменшує дублювання логіки та дозволяє розглядати PickupItem як проміжний компонент між фізичним об'єктом сцени й системою обліку предметів.

3.3.2 Реалізація взаємодії з предметами та інвентарної логіки

Наявний фрагмент коду дає змогу проаналізувати базовий сценарій підбору предмета в середовищі Unity. Клас PickupItem реалізовано як компонент MonoBehaviour, який прикріплюється до об'єкта сцени та містить посилання на ItemScriptableObject, а також числовий параметр amount. Це означає, що об'єкт у сцені має не лише візуальне представлення, але й програмну поведінку, пов'язану з переміщенням предмета до інвентарю. У такій структурі сам GameObject виступає носієм інтерактивної поведінки, а ScriptableObject — джерелом описових даних [4; 6].

Метод Pickup() реалізує послідовний алгоритм безпечного виконання операції підбору. На початковому етапі він перевіряє, чи призначено об'єкт item, оскільки без опису предмета неможливо коректно додати його до

інвентаря. Якщо поле елемента не заповнене, метод виводить повідомлення про помилку через `Debug.LogError()` та завершує виконання без зміни стану сцени. Така перевірка запобігає ситуації, коли об'єкт буде знищено без фактичного додавання предмета до системи інвентаря [14].

Після перевірки предметних даних метод виконує пошук `InventoryManager` у сцені. Цей менеджер діє як центральний компонент, відповідальний за зміну стану запасів та прийняття замовлень від `PickupItem`. Якщо менеджер не знайдено, метод також завершується з повідомленням про помилку. Лише після успішного знаходження `InventoryManager` викликається метод `AddItem(item, amount)`, який передає до системи інвентаря опис предмета та його кількість.

Після успішного додавання предмета до інвентаря об'єкт видаляється зі сцени за допомогою `Destroy(gameObject)`. Це відображає зміну стану ігрового світу: предмет більше не повинен бути доступним для повторного отримання після того, як його було враховано в інвентарі. Таким чином, метод `Pickup()` одночасно виконує перевірку даних, взаємодію з менеджером інвентаря, журналювання події та оновлення сценового стану. Така послідовність забезпечує логічну завершеність сценарію підбору.

Архітектурно клас `PickupItem` не зберігає стан інвентаря самостійно. Він лише ініціює операцію додавання, делегуючи зміну стану компоненту `InventoryManager`. Такий розподіл узгоджується з принципом індивідуальної відповідальності, де `PickupItem` відповідає за поведінку певного елемента сцени, а `InventoryManager` відповідає за підрахунок елементів у системі. Це спрощує подальше розширення функціоналу, оскільки зміни у правилах зберігання інвентаря не обов'язково потребуватимуть зміни кожного сценового предмета.

Разом із тим використання `FindObjectOfType()` має технічні обмеження. У сучасній документації Unity цей метод позначено як застарілий, він повертає перший активний завантажений об'єкт відповідного типу та не рекомендований для частого використання [13]. За даного сценарію цей пошук може бути прийнятним як відповідь на одну процедуру вибору; однак, для більш надійної структури доцільно використовувати попередньо визначене посилення, централізовану конфігурацію або більш сучасні методи пошуку об'єктів. Окремо потрібно перевірити назву `item.itemName`, оскільки вона може містити помилку іменування або відповідати конкретній структурі `ItemScriptableObject`, код якого не наведено.

3.3.3 Інтеграція асетів та побудова сцен

Інтеграція зовнішніх асетів у Unity повинна розглядатися не лише як додавання візуальних моделей, а як частина технічної організації сцени. Prefab-система Unity дозволяє зберігати `GameObject` разом із компонентами, дочірніми об'єктами та значеннями властивостей як повторно використовуваний шаблон [7]. Це надзвичайно важливо для дверей, предметів для збирання, елементів навколишнього середовища та інших речей, які можна

повторно використовувати в різних частинах сцени. У такому разі один ресурс може використовуватися в кількох місцях без дублювання налаштувань.

Під час імпорту асетів необхідно враховувати не лише їхній зовнішній вигляд, а й технічну сумісність із версією Unity та обраним render pipeline. Якщо пакет був створений або протестований для іншої версії движка або іншої системи відображення, може знадобитися додаткове налаштування матеріалів, тіней та освітлення. Тому аналіз сумісності ресурсів є важливою частиною підготовки стабільної сцени. У курсовій роботі такі відомості доцільно подавати окремо від опису фактичного використання об'єктів у проєкті.

Наведена таблиця відокремлює підтверджені службові дані пакетів від припущень щодо їх використання. Такий підхід важливий для академічної точності, оскільки сторінка магазину ресурсів підтверджує функції пакета, але не демонструє, як інтегрувати їх у конкретний студентський проєкт. Тому в остаточній редакції курсової роботи потрібно додати скріншоти сцени, Hierarchy, Inspector або Project window, які підтверджують реальне використання відповідних ресурсів.

Наявний перелік асетів свідчить про потенційне поєднання різних візуальних напрямів. Abandoned Asylum може формувати реалістичніше занедбане інтер'єрне середовище, тоді як 3D Low Poly Magical Forest і POLY - Lite Survival Collection мають стилізований характер. Якщо ці ресурси використовуються в одному проєкті, важливо забезпечити узгодженість масштабу моделі, освітлення, матеріалів, колізій та загального художнього стилю. Без такого узгодження сцена може втратити цілісність і виглядати як набір випадково об'єднаних пакетів.

Цей рисунок доцільно розмістити після опису інтеграції асетів. Він повинен виконувати функцію доказу, тобто підтверджувати не лише існування пакетів у проєкті, але й їх використання у сцені або структурі ресурсів. У тексті курсової роботи після вставлення рисунка потрібно коротко пояснити, які саме об'єкти на ньому показано та яку роль вони виконують у побудові середовища.

3.3.4 Користувацький інтерфейс, збереження даних і стабільність роботи

У наданих матеріалах не подано окремих скриптів, макетів або скріншотів користувацького інтерфейсу. Через це неможливо достовірно встановити склад екранів, панелей, кнопок, слотів інвентаря або сценаріїв навігації. Водночас, наявність InventoryManager у логіці вибору товарів вказує на те, що інтерфейс має бути пов'язаний з відображенням даних про запаси або статусу агрегованих товарів. Тому в цій частині можна описувати лише зв'язок між підбором предмета та потенційним оновленням інвентаря, але не конкретну реалізацію UI без додаткових матеріалів.

Якщо в проєкті використовується UI Toolkit, опис інтерфейсу доцільно будувати через розділення структури, стилю та поведінки. У цьому випадку UXML може бути відповідальним за візуальну деревоподібну структуру, USS

за форматування, а код C# за обробку подій та передачу даних. Якщо ж інтерфейс реалізовано через Canvas, потрібно описувати ієрархію UI-елементів, прив'язку кнопок і взаємодію зі скриптами. За відсутності підтверджених UI-ресурсів остаточний перелік елементів інтерфейсу слід уточнити окремо [10; 11].

Підсистема збереження даних у наведеному коді не представлена. Таким чином, не можна стверджувати, що в проєкті фактично реалізовано збереження інвентарю, збереження прогресу або налаштування користувача. З технічної точки зору статичні описи предметів доцільно відокремлювати від змінного стану гри, оскільки ScriptableObject зручно використовувати як джерело довідкових даних, але не як основний механізм збереження поточного ігрового сеансу [6]. Для фактичного опису збереження необхідно надати відповідний код або структуру файлів.

Якщо в подальшій реалізації використовується файлове збереження, логічним підходом є застосування Application.persistentDataPath як каталогу для даних, які мають зберігатися між запусками застосунку [16]. Для серіалізації стану може використовуватися JsonUtility.ToJson, який формує JSON-представлення об'єкта на основі полів, що відповідають правилам серіалізації Unity. Однак у поточних матеріалах такого коду немає, тому ці механізми можна згадати лише як можливий напрямок впровадження, а не як підтверджені функції. Це дає змогу уникнути необґрунтованого опису системи збереження.

Стабільність роботи в компоненті PickupItem забезпечується передусім перевіркою критичних залежностей до виконання основної операції. Якщо не призначено item або в сцені відсутній InventoryManager, метод не продовжує виконання й повідомляє про помилку через Debug.LogError() [14]. Ця логіка зменшує ризик неправильного знищення елемента та допомагає швидше виявляти помилки конфігурації в Інспекторі або сцені. У результаті компонент поводить себе безпечніше, ніж реалізація, яка одразу знищувала б об'єкт без перевірки стану системи.

Для повнішого опису стабільності потрібно уточнити такі аспекти:

- чи існує в проєкті окремий UI-модуль інвентаря;
- чи оновлюється інтерфейс після виклику InventoryManager.AddItem();
- чи реалізовано збереження інвентарних даних між ігровими сесіями;
- чи є в InventoryManager перевірка максимальної кількості предметів або повторного додавання;
- чи передбачено захист від некоректного значення amount.

Ці пояснення необхідні для уточнення не лише локальної логіки функції PickupItem, але й повної поведінки системи інвентаризації. Без відповідного коду неможливо зробити висновки щодо правил обмеження інвентаризації, форматів зберігання або реакції інтерфейсу на зміни даних. Тому цей розділ слід розглядати як підтвердження базового сценарію отримання, а не як повну реалізацію підсистеми інвентаризації.

Ця діаграма має підтвердити налаштування компонента в редакторі Unity. Доцільно використовувати її як доказ того, що дані елемента дійсно пов'язані з об'єктом сцени, а не просто описані в коді. Після вставки діаграми слід коротко пояснити заповнені поля компонента, а також те, як вони впливають на виконання функції `Pickup()`.

3.3.5 Документування коду та тестування функціоналу

Якість супроводу Unity-проєкту залежить від зрозумілого документування вихідного коду та відтворюваності сценаріїв перевірки. У C# для цього можуть використовуватися XML-коментарі, які описують призначення класів, методів, параметрів і результатів виконання [18]. У цій частині доцільно задокументувати призначення класу `PickupItem`, вміст полів `Item` та `Amount`, передумови для виклику методу `Pickup()` та залежність від `InventoryManager`. Таке документування особливо важливе в Unity-проєктах, де частина логіки задається не лише кодом, а й налаштуваннями `Inspector`.

Метод `Pickup()` має чіткі передумови виконання. По-перше, поле `item` повинно містити посилання на коректний `ItemScriptableObject`. По-друге, у сцені повинен існувати активний `InventoryManager`. По-третє, значення `amount` повинно відповідати очікуваній кількості предметів, яка додається до інвентаря. Якщо ці умови виконуються, метод додає елемент до інвентарю, записує подію в консолі та видаляє об'єкт сцени.

Базові сценарії тестування для цього функціоналу можна сформулювати так:

1. Сценарій успішного підбору предмета перевіряє наявність `ItemScriptableObject`, доступність `InventoryManager`, виклик `AddItem(item, amount)` і зникнення предмета зі сцени.
2. Сценарій відсутності опису предмета перевіряє, що за `item == null` метод завершується достроково, виводить повідомлення про помилку й не знищує `GameObject`.
3. Сценарій відсутності `InventoryManager` перевіряє, що предмет не додається до інвентаря та не видаляється зі сцени, якщо менеджер не знайдено.
4. Сценарій перевірки кількості предметів має встановити, чи коректно обробляється значення `amount` і чи не допускаються некоректні значення.
5. Сценарій інтеграції ресурсів сцени має перевірити `prefab`-налаштування, колізії, точки взаємодії та відповідність використаних асетів обраній конфігурації `render pipeline`.

Наведені сценарії охоплюють як локальну роботу компонента `PickupItem`, так і його взаємодію з інвентарною підсистемою. Його можна використовувати як основу для ручного тестування або для створення тестів у режимі редагування та запуску в рамках `Unity Testing Framework`. Водночас у наданих матеріалах відсутні фактичні журнали виконання тестів, таблиці

результатів або скріншоти Test Runner, тому робити висновки про успішність тестування поки що некоректно.

Unity Test Framework підтримує поділ перевірок на Edit Mode і Play Mode тести [19; 20]. Edit Mode тести доцільні для перевірки редакторної або допоміжної логіки, тоді як Play Mode тести можуть використовуватися для сценаріїв, що потребують запуску runtime-поведінки. Для поточного ігрового режиму тести будуть доречними, оскільки вибір предметів, пошук в менеджері інвентарю та видалення ігрових об'єктів відбуваються в контексті сцени. Проте фактичне використання Unity Test Framework у проєкті потрібно підтвердити окремими матеріалами.

Підсумовуючи, наданий код підтверджує реалізацію базового сценарію підбору предмета та його передачі до інвентарної системи. Найбільш специфічними елементами є клас PickupItem, виклик ItemScriptableObject, пошук InventoryManager, виклик AddItem() та видалення об'єкта після успішної операції. Менш визначеними залишаються внутрішня структура InventoryManager, будова ItemScriptableObject, інтерфейс інвентаря, система збереження даних і результати тестування. Тому розділ можна використовувати як основу для курсової роботи, але перед остаточним поданням його слід доповнити скріншотами, кодом пов'язаних класів і фактичними результатами перевірки.

3.4 Доказова методологія підготовки фрагментів курсової роботи для Unity-проєкту

3.4.1 Межі завдання та принципи доказовості

Специфікація, орієнтована на створення фрагментів курсової роботи за кодом, скріншотами та асетами Unity, є методологічно коректною лише за умови суворого розмежування між спостережуваними фактами та їх інтерпретацією. У середовищі Unity технічна реальність проєкту в першу чергу відображається у сценах, структурі GameObject, компонентах, послідовних полях, попередньо створених моделях та метаданих пакета. Офіційна документація визначає сцени як assets, що містять увесь або частковий вміст гри чи застосунку, а GameObject — як базову сутність, яка може існувати в сцені та виступає контейнером для функціональних компонентів [3; 4]. Відповідно, академічний опис повинен відтворювати саме підтверджені рівні системи, а не декларувати архітектурні властивості, яких немає в коді, сцені або супровідних матеріалах.

Особливої уваги потребує версіонованість Unity-екосистеми. Сумісність ресурсів та пакетів залежить не лише від назви постачальника, але й від конкретної версії редактора, активного конвеєра постачання та технічних параметрів імпортованого пакета. Якщо сторінка Unity Asset Store містить відомості про версію, дату релізу, розмір, тип ліцензії або сумісність із Built-in, URP чи HDRP, ці дані потрібно фіксувати в курсовій роботі як службову технічну інформацію. Такий підхід дає змогу уникнути нечіткого

формулювання на кшталт «асет використано для сцени» без пояснення, у якому середовищі та з якими обмеженнями він застосовується [21; 22].

З огляду на це, вимога не вигадувати архітектуру, функціонал або результати тестування є не лише редакторським обмеженням, а технічно необхідним правилом. У проєкті Unity та сама сцена може поводитися по-різному після міграції між версіями редактора, зміни системи введення, перемикання на інший шлях або імпорту іншої версії готової моделі. Тому кожне твердження в тексті курсової роботи доцільно пов'язувати з одним із чотирьох видів джерельної опори: кодом, візуальним доказом зі скріншота, метаданими пакета або результатом тесту. Якщо такої опори немає, коректніше залишити формулювання про потребу уточнення, ніж реконструювати відсутні факти.

Наведену логіку доцільно формалізувати як послідовність переходу від матеріалів проєкту до готового академічного тексту:



Рисунок 3 – Процес трансформації матеріалів Unity-проєкту в академічний фрагмент курсової роботи

Подана схема показує, що фрагмент курсової роботи повинен формуватися не з припущень, а з послідовного аналізу матеріалів проєкту. [На першому рівні фіксуються технічні артефакти, на другому — їх інтерпретація, а на третьому — академічно оформлений текст.] Така послідовність дозволяє

зберігати доказовість і не підмінювати реальну структуру Unity-проєкту типовими описами.

3.4.2 Технологічна база Unity та C#

Архітектурне ядро Unity є компонентно-орієнтованим. `GameObject` сам по собі не реалізує прикладну поведінку, а виступає контейнером для компонентів, які визначають зовнішній вигляд, фізичну поведінку, логіку взаємодії, UI-представлення або сервісні сценарії [4]. Кожен елемент гри містить трансформацію, а інші компоненти додаються відповідно до функціонального призначення елемента. Для курсової роботи це означає, що опис реалізації доцільно будувати не навколо абстрактних «монолітних модулів», а навколо сцени, префабів, компонентів і ролей окремих C#-класів у цій компонентній моделі.

Сценарії прикладної поведінки в Unity зазвичай реалізуються через класи, що наслідують `MonoBehaviour`. Документація Unity визначає `MonoBehaviour` як базовий клас, від якого успадковується багато Unity-скриптів, і підкреслює, що такі об'єкти існують як компоненти `GameObject` [5]. Тому, описуючи код, важливо пояснити не лише вміст класу, але й до якого об'єкта сцени він може бути приєднаний та яку роль він відіграє в життєвому циклі сцени. Це дає змогу пов'язати вихідний код із фактичним функціонуванням програмного продукту.

Не менш важливим є рівень серіалізації, оскільки саме він визначає, які дані реально конфігуруються через `Inspector`. В Unity послідовні поля дозволяють переносити певні налаштування з коду до редактора, що спрощує зміну параметрів без безпосереднього редагування скриптів. `ScriptableObject`, у свою чергу, використовується як тип для збереження даних незалежно від конкретного екземпляра `GameObject`, що робить його доречним для опису предметів, конфігурацій або довідкових сутностей [6]. У тексті курсової роботи такі механізми слід описувати лише тоді, коли вони підтверджені кодом, структурою `assets` або скріншотами `Inspector`.

Для модульності коду Unity надає механізм `Assembly Definition`, тобто `.asmdef`-файлів. Офіційна документація вказує, що організація скриптів у власні збірки дає змогу явно задавати залежності та розмежовувати частини коду [8]. Це має особливу цінність для опису навчального курсу, оскільки наявність файлу `.asmdef` є фактичною, а не просто декларативною ознакою стандартної організації. Якщо в проєкті таких файлів немає, формулювання про «чітко ізольовані модулі» потребує підтвердження через інші артефакти, наприклад структуру тек, простори імен або схему залежностей.

На рівні мови C# академічний аналіз доцільно прив'язувати до конкретних механізмів об'єктно-орієнтованого програмування. Інтерфейси задають контракти поведінки, модифікатори доступу визначають межі використання типів і членів, а `virtual/override` підтримують поліморфне розширення поведінки [17]. Це інструменти, які слід використовувати як мову для опису архітектури в робочому тексті, а не обмежуватися загальним твердженням про

використання об'єктно-орієнтованого програмування. Якщо код не містить інтерфейсів, успадкування або перевизначення методів, такі особливості не слід приписувати проєкту без доказів.

Практично значущими одиницями опису, які легко верифікуються у Unity-проєкті, є такі сутності:

- класи `MonoBehaviour`, прив'язані до `GameObject` і сценової логіки;
- префаби як шаблони повторно використовуваних об'єктів;
- `ScriptableObject` як носії конфігураційних або довідкових даних;
- `.asmdef`-файли як маркери модульних меж і явних залежностей;
- інтерфейси та базові класи як формалізовані контракти взаємодії.

Наведений перелік задає основу для доказового опису архітектури. Кожну сутність можна підтвердити певним елементом: файлом коду, файлом ресурсів, знімком екрана з програми `Inspector` або структурою проєкту. Завдяки цьому курсова робота не перетворюється на загальний опис Unity, а залишається прив'язаною до фактичної реалізації.

3.4.3 Аналіз шаблонного компонента `OpenableObject`

Наданий шаблон, який включає `Directions` та `OpenableObject`, ілюструє, як описати навіть короткий код у дослідницькій роботі без додавання зайвої логіки. Перерахування `Directions` у C# визначає фіксований набір іменованих констант, що спрощує визначення параметрів для відкриття або взаємодії з напрямками. Клас `OpenableObject` успадковується від `MonoBehaviour` і, таким чином, функціонує як плагін, який можна додати до `GameObject` у сцені Unity. Поле `_openOrCloseTime`, позначене властивістю `SerializeField`, ілюструє явну серіалізацію неpubлічного поля, що дозволяє змінювати параметри через `Inspector` без відкриття прямого публічного доступу.

Основна функціональність цього класу спирається на підпрограми. У Unity підпрограма дозволяє призупинити та відновити функцію пізніше, розподіляючи роботу між кількома кадрами. У наданому шаблоні функція `OpenOrClose` спочатку скасовує поточні підпрограми, а потім, на основі прапорця `_isOpen`, виконує `Close()` або `Open()`. Це дозволяє нам інтерпретувати компонент як базовий елемент керування для періодичного руху стану, тобто як основу для логіки відкриття та закриття об'єкта.

Водночас, фрагмент коду в його поточному вигляді не реалізує повну поведінку руху. Функції `Open()` та `Close()` очікують лише зміни значення `_openOrCloseLerp`, але вони не змінюють це поле незалежно в межах заданого коду. Тому з академічної точки зору неправильно стверджувати, що клас виконує плавне відкриття дверей, якщо він не надає додаткових похідних класів, обертання `Transform` або зміни локального положення. Правильне формулювання полягає в тому, що клас визначає базовий вузол для відкритого стану об'єкта, а повний механізм руху має бути реалізований у перевизначених функціях за замовчуванням або в іншому фрагменті коду, який можна стверджувати.

З методологічної точки зору, цей приклад чітко ілюструє, як описувати принципи об'єктно-орієнтованого програмування в дослідницькій роботі. Наявність функцій за замовчуванням є прямим показником поліморфної розширюваності, оскільки похідні класи можуть перевизначати базову поведінку. Використання слова «protected» для полів `_openOrCloseTime`, `_openOrCloseLerp` та `_isOpen` вказує на те, що автор коду дозволяє їх використання в ієрархії успадкування, але не робить їх доступними як частину зовнішнього API. Тому в цій роботі має сенс обговорити використання інкапсуляції та успадкування, а не існування конкретних підкласів, якщо їхній код не надано.

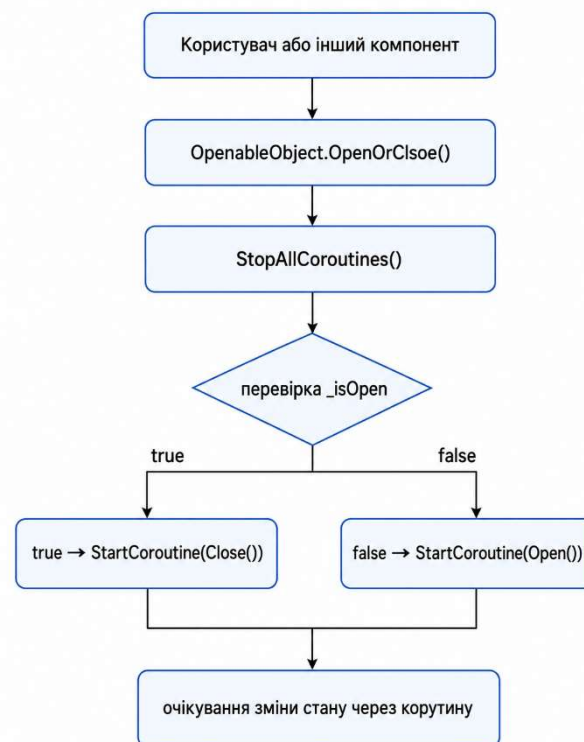


Рисунок 4 – Логіка роботи базового відкриваного об'єкта

На діаграмі показано, що `OpenableObject` не є повноцінною системою дверей, а радше базовим компонентом керування станом. Його функція полягає в регулюванні переходу між відкритим та закритим станами, тоді як будь-який рух, обертання або зміна положення мають бути підтверджені окремим кодом. Цей опис зберігає технічну точність і не приписує класу функції, які не очевидні з наданого розділу.

3.4.4 Архітектура модулів, інтерфейсу та збереження даних

Під час опису користувацького інтерфейсу в Unity доцільно чітко відрізнити підтверджені технології від можливих альтернатив. UI Toolkit офіційно розглядається як система інструментів для створення інтерфейсу користувача, зокрема runtime UI [10; 11]. Якщо скріншоти або код показують `UIDocument`, `UXML` і `USS`, це є достатньою підставою описувати інтерфейс як

декларативно-компонентний. За відсутності таких доказів не слід автоматично приписувати проєкту UI Toolkit, оскільки інтерфейс може бути реалізований через Canvas або іншу схему.

Схожа ситуація стосується системи введення. Якщо в коді використано `Input.GetKeyDown` або інші виклики класичного Input API, це потрібно описувати як роботу з відповідним механізмом Unity, а не як автоматичне використання нового Input System Package [12]. Для академічного опису важливо розвести шар введення, шар ігрової логіки та шар UI-взаємодії. Таке розмежування дозволяє зрозуміти, чи є введення жорстко прив'язаним до клавіш, чи реалізоване через дії та прив'язки.

Для підрозділів, присвячених збереженню даних, доцільно спиратися на чітко відмінні механізми Unity, а не об'єднувати їх у загальне поняття «система сейвів». `ScriptableObject` придатний для зберігання конфігураційних або довідкових даних на рівні asset-файлів [6]. `PlayerPrefs` зберігає між сесіями лише значення простих типів, зокрема `string`, `float` та `int`, і не повинен описуватися як універсальна система збереження складного ігрового стану [15]. Для структурованих станів застосунку природнішим підходом є зв'язка `JsonUtility` і `Application.persistentDataPath`, однак її можна описувати як реалізовану лише за наявності відповідного коду.

Механізм	Підтверджені можливості	Обмеження, які слід зафіксувати в тексті
<code>ScriptableObject</code>	Зберігає дані як asset і може використовуватися для конфігураційних або довідкових сутностей	Не є автоматичним доказом реалізації runtime-збереження поточного стану гри
<code>PlayerPrefs</code>	Зберігає прості значення між ігровими сесіями	Не підходить для складних структур і чутливих даних
<code>JsonUtility</code> + <code>Application.persistentDataPath</code>	Може використовуватися для JSON-серіалізації та запису змінних даних у стабільний каталог	Потребує підтвердження кодом і дотримання правил серіалізації Unity

У таблиці показано, що різні механізми збереження служать різним цілям. Тому у вашому дослідженні ви не можете розглядати жоден з них як повноцінну систему збереження без підтвердження їх фактичного впровадження. Якщо код збереження не надано, вам слід лише описати можливі напрямки впровадження або вказати на необхідність уточнення.

Додатково варто враховувати, що `Application.persistentDataPath` призначений для даних, які мають зберігатися між запусками застосунку [16]. Якщо змінний ігровий стан зберігається у файлах, саме такий каталог є

логічним місцем для запису даних. Проте для академічного опису недостатньо просто згадати цей шлях: потрібно показати код запису, читання, обробки помилок і структуру даних, що серіалізуються. Без цього твердження про реалізовану систему збереження буде неповним.

3.4.5 Забезпечення стабільності, документування й тестування

Опис стабільності Unity-проєкту має спиратися на конкретні механізми контролю помилок, а не на загальні фрази про «надійність системи». На рівні Unity для діагностики можуть застосовуватися `Debug.Log`, `Debug.LogWarning` і `Debug.LogError`, причому `Debug.LogError` фіксує повідомлення про помилку в консолі [14]. У курсовій роботі такі виклики доцільно описувати як засоби виявлення проблем конфігурації або некоректних станів, а не як повноцінну систему обробки винятків. Для модулів, які працюють із файлами або зовнішніми даними, додатково може знадобитися `try/catch`, але це має підтверджуватися кодом.

Правильне розуміння підпрограм є вирішальним аспектом стабільності. Підпрограма — це спосіб розподілу часу виконання між кадрами, але її не слід описувати як окремий потік або автоматизований механізм багатопроцесорної обробки. Це означає, що підпрограми підходять для прогресивної анімації, очікування подій, пауз та часових переходів, але вони не використовуються для інтерпретації складних обчислень, які перешкоджають виконанню. Ця тонка відмінність має бути чітко зазначена в тексті курсу; інакше опис буде технічно неточним.

Для документування коду в середовищі C# доцільно спиратися на XML documentation comments. Microsoft визначає потрібний слеш `///` як стандартний механізм створення документаційних коментарів, а рекомендовані теги `summary`, `param`, `returns`, `remarks` та `inheritdoc` допомагають підтримувати узгоджений рівень API-документації [18]. У контексті курсової роботи наявність таких коментарів можна описувати як елемент супровідної документації коду лише тоді, коли вони справді присутні у файлах проєкту. Якщо коментарів немає, коректно зазначити, що документування здійснюється через структуру класів, назви членів і пояснювальний текст роботи.

Щодо тестування, Unity Test Framework є офіційним інструментом верифікації коду в Unity-проєктах. Він підтримує Edit Mode і Play Mode тести, що дозволяє розділяти перевірки допоміжної логіки та runtime-поведінки сцени [19; 20]. Якщо результати таких тестів у матеріалах проєкту не надані, у тексті курсової роботи не слід вигадувати відсотки покриття, сценарії проходження або статистику успішності. У такому разі можна описати рекомендований підхід до перевірки, але не фактично отримані результати.

З практичного погляду перевірка Unity-проєкту може мати двошарову структуру:

- на рівні Edit Mode доцільно перевіряти серіалізацію, допоміжні класи, перетворення даних, контракти інтерфейсів і обчислювальну логіку;
- на рівні Play Mode доцільно перевіряти поведінку MonoBehaviour, корутин, реакцію на введення, роботу UI та взаємодію з фізикою;
- за наявності журналювання можна окремо перевіряти очікувані повідомлення в консолі;
- для фінального розділу курсової потрібно подавати не лише сценарії, а й фактичні результати їх виконання.

Цей список не є твердженням про те, які тести фактично були проведені, а радше систематичною структурою, яку можна використовувати після надання розкладів тестування, знімків екрана тестового драйвера або журналів виконання. Такий підхід дозволяє зберегти академічну точність і уникнути враження, що є тести, які не були підтверджені в матеріалах проекту.

3.4.6 Інтеграція сторонніх асетів і ліцензійні обмеження

Сторонні асети в Unity-проекті не є другорядними декоративними елементами, а повноцінними технічними залежностями. Вони впливають на архітектуру сцен, відтворюваність результатів, сумісність із версією Unity та коректність опису проекту. Саме тому в тексті курсової роботи варто фіксувати не лише назву пакета, а й версію, дату останнього релізу, вихідну версію Unity, тип ліцензії та сумісність із render pipeline. Це переводить згадку про асет із рівня неформального переліку ресурсів на рівень технічної специфікації [21; 22].

Пакет	Підтверджені метадані	Значення для курсового опису
Abandoned Asylum	Standard Unity Asset Store EULA; license type — Extension Asset; file size — 1.8 GB; latest version — 1.4; latest release date — Jul 17, 2020; original Unity version — 2019.4.1	Може бути описаний як зовнішній пакет оточення з фіксованою версією та відомою вихідною версією редактора
3D Low Poly Magical Forest	Standard Unity Asset Store EULA; file size — 5.7 MB; latest version — 1.1.0; latest release date — May 13, 2026; original Unity version — 2022.3.26	Показує залежність контенту від версії Unity і render pipeline, що слід прямо відображати в тексті
Free Wood Door Pack	Standard Unity Asset Store EULA; file size — 99.3 MB; latest version — 1.0; latest release date	Доцільний як приклад зовнішнього пакета дверей із зафіксованою

	— Apr 22, 2024; original Unity version — 2021.3.32	версією та технічними метаданими
POLY - Lite Survival Collection	Standard Unity Asset Store EULA; file size — 5.7 MB; latest version — 1.1.0; latest release date — May 13, 2026;	Показує залежність контенту від версії Unity і render pipeline, що слід прямо відображати в тексті

Наведений вище запис дозволяє вам документувати сторонні ресурси в обґрунтованій академічній манері. Він показує не лише назву пакета, але й важливі технічні параметри для відтворення проекту. Однак наявність сторінки сховища ресурсів не доводить, що всі елементи пакета використовуються в конкретному проекті, тому фактичну інтеграцію необхідно підтвердити через проект, ієрархію, скріншоти інспектора або екземпляри сцен.

У ліцензійному аспекті потрібно враховувати, що Unity Asset Store EULA регулює використання асетів як ліцензованих ресурсів [21]. Довідка Unity також розмежовує Extension Assets, Single Entity та Multi Entity assets, що має значення для документування правового статусу використаних пакетів [22]. Для курсової роботи це означає, що асети слід описувати як інтегровані сторонні залежності проекту, а не як повністю авторсько створений контент. Якщо пакет було модифіковано, адаптовано або перефарбовано, такий факт також потрібно підтверджувати матеріалами проекту.

Детальні специфікації контенту, такі як кількість кімнат, моделей, текстур, готових елементів або варіацій кольорів, потребують особливої уваги. Якщо офіційна картка пакета, локальна документація або структура імпортованого файлу не підтверджують ці критерії, їх не слід представляти як факти. Точніше, необхідно використовувати додаткові докази для остаточного опису компонентів пакета. Це відповідає цілісності пошуку та запобігає заміні технічного аналізу описами рекламних ресурсів.

3.4.7 Практичні висновки для формування академічного тексту

Огляд показує, що запропоновані специфікації забезпечують міцну основу для розробки навчальних матеріалів, але їхня валідність залежить від їхньої кореляції з конкретними компонентами проекту Unity. Оптимальна модель — це та, де кожен розділ контенту базується або на вихідному коді, візуалізації сцени, метаданих пакета, або на результаті тестування. Це узгоджується з компонентною природою Unity, його системою нумерації версій для вікон перегляду, механізмами введення, системами послідовності та тестовими пакетами. Відсутність цієї кореляції збільшує ймовірність хибних архітектурних функцій, особливо в розділах, пов'язаних з інтерфейсом користувача, сховищем даних та результатами тестування продуктивності.

У практичному застосуванні такої специфікації доцільно дотримуватися кількох опорних правил:

1. Фіксувати у тексті точну версію Unity, активний render pipeline і зовнішні пакети.
2. Описувати класи через їхню архітектурну роль: компонент, сервіс, модель даних, тест або конфігураційний asset.
3. Відокремлювати факти реалізації від припущень про намір автора коду.
4. Не прирівнювати наявність корутин до багатопоточності, а PlayerPrefs — до повноцінної системи сейвів.
5. Позначати непідтверджені твердження як такі, що потребують уточнення або додаткового джерела.

Ці правила допомагають стандартизувати методологію написання різних розділів дослідницької роботи. Вони особливо важливі, коли текст пишеться на основі фрагментів коду, неповних скріншотів або окремих елементів, а не повного проєкту. Це гарантує, що кінцевий матеріал залишається академічним, технічно точним та адаптованим для подальшого використання в конкретних застосунках.

На завершення, добре досліджені специфікації є найефективнішими, коли ви розглядаєте проєкт Unity як сукупність задокументованих технічних елементів. З наукової та методологічної точки зору, високоякісна дослідницька робота повинна демонструвати не лише формальний академічний стиль, але й точність джерела на рівні версії рушія, включаючи функції пакета, послідовні структури, взаємодію компонентів та результати тестування. Ця модель дозволяє перетворити фрагменти коду, знімки екрана та ресурси на вичерпний академічний опис без заміни аналізу припущеннями.

3.5 Архітектура та компоненти програмного продукту

3.5.1 Архітектура програмного продукту та компонентна структура Unity-проєкту

Архітектура програмного продукту побудована відповідно до компонентної моделі Unity, у якій базовою одиницею сцени є `GameObject`, а функціональність об'єкта визначається набором прикріплених до нього компонентів [4]. У межах проєкту поведінка ігрових об'єктів реалізується через C#-скрипти, що використовуються як окремі програмні модулі та відповідають за визначені функції системи. Такий підхід дає змогу розділити відповідальність між частинами програмного продукту й уникнути надмірного поєднання логіки інвентаря, взаємодії, інтерфейсу та збереження даних в одному компоненті. Завдяки цьому структура проєкту залишається зрозумілою для подальшого супроводу, розширення та аналізу в межах курсової роботи.

У проєкті застосовано розділення статичних даних і функціональної поведінки. Дані предметів винесено в `ScriptableObject`-клас, тоді як логіка роботи з об'єктами сцени реалізується у `MonoBehaviour`-скриптах, прикріплених до відповідних `GameObject`. `ScriptableObject` використовується як контейнер спільних даних, що дозволяє зберігати характеристики предметів

незалежно від конкретних екземплярів у сцені та зменшувати дублювання однакових значень у різних об'єктах. Така організація відповідає загальній логіці Unity, де дані, поведінка та візуальне представлення можуть бути розділені між різними типами об'єктів і компонентів [5], [6].

Архітектура програмного продукту передбачає взаємодію між модулями управління запасами, інтерфейсом користувача, елементами продукту та ресурсами, що зберігають метадані. Кожен компонент відіграє окрему роль: деякі елементи містять дані, інші їх відображають, а окремі скрипти обробляють дії користувача. Отже, система не обмежується одним інтегрованим скриптом, а складається з взаємопов'язаних елементів, кожен з яких має певне призначення. Це покращує управління проектами та спрощує процес перевірки різних компонентів реалізації.

3.5.2 Структура даних предметів

У системі предметів реалізовано клас `ItemScriptableObject`, який призначений для зберігання статичних характеристик ігрових предметів. Він успадковується від `ScriptableObject`, тому його екземпляри можуть існувати як окремі ресурси проєкту, а не лише як компоненти сценових об'єктів [6]. Основна роль цього класу полягає в централізованому описі предмета: його назви, префаба, іконки, текстового опису, типу, максимальної кількості в стеку та можливих ефектів споживання. Завдяки цьому інші компоненти можуть отримувати інформацію про предмет через посилання на один ресурс, не дублюючи однакові дані в кожному екземплярі об'єкта.

У наступній таблиці показано структуру класу, який описує основні властивості елемента. Поле `itemName` зберігає назву елемента, а поле `itemPrefab` містить посилання на відповідний попередньо створений шаблон. Код використовується для відображення елемента в інтерфейсі користувача. Поле `itemDescription` використовується для текстового опису, а атрибут `TextArea` полегшує редагування багаторядкового тексту у вікні Інспектора. Атрибути `Header` логічно групують поля в редакторі Unity, що робить структуру ресурсів зрозумілішою під час ініціалізації елементів.

Перелік `ItemType` використовується для класифікації предметів за типом. Поточна структура надає такі значення: `Default`, `Food`, `Weapon` та `Gadget`, що дозволяє розрізняти предмети за їхньою функцією в системі. Поле `MaximumAmount` визначає максимальну кількість одиниць одного предмета в ігровому інвентарі, що важливо для контролю накопичення однакових предметів. Тег `isConsumable` та параметри `healthChange`, `hungryChange` та `thirstChange` описують потенційні зміни стану персонажа після використання предмета, якщо предмет є витратним.

Поле	Призначення
<code>itemName</code>	Назва предмета, що використовується для ідентифікації в інтерфейсі та логіці.

itemPrefab	Префаб об'єкта сцени, який відповідає конкретному предмету.
icon	Спрайт для візуального відображення предмета в UI.
itemDescription	Текстовий опис предмета для інформаційних панелей або підказок.
itemType	Текстовий опис предмета для інформаційних панелей або підказок.
maximumAmount	Максимальна кількість одиниць предмета в одному стеку.
isConsumable	Ознака того, чи може предмет бути спожитий.
healthChange	Зміна показника здоров'я після використання предмета.
hungerChange	Зміна рівня голоду після використання предмета.
thirstChange	Зміна рівня спраги після використання предмета.

Подана таблиця узагальнює структуру класу та демонструє, що `ItemScriptableObject` не виконує активну поведінку, а зберігає дані, необхідні іншим частинам системи. Наприклад, під час підбору предмета компонент взаємодії може передати в інвентар посилання на відповідний ресурс, після чого інтерфейс використовує його назву, іконку та опис. Таким чином, клас виконує роль інформаційної основи для системи предметів і забезпечує єдине джерело даних для різних модулів.

На діаграмі показано, що `ItemScriptableObject` успадковується від `ScriptableObject` та використовує перерахування `ItemType` для класифікації елементів. Вона також показує зв'язок між контейнером даних та типом елемента, що використовується для обробки елементів у системі інвентаризації. Ця структура дозволяє елементу описувати один ресурс, який використовується як у системній логіці, так і в інтерфейсі користувача.

3.5.3 Модульність і взаємодія компонентів

Модульність проєкту забезпечується розподілом функцій між окремими класами та компонентами Unity. `GameObject` виступає контейнером, до якого можуть бути додані різні компоненти, а `MonoBehaviour`-скрипти дають змогу реалізувати поведінку об'єктів у сцені [4], [5]. Завдяки цьому кожний функціональний блок системи може бути описаний як окремий модуль, що має власне призначення та взаємодіє з іншими частинами проєкту через посилання, методи або події. Така організація полегшує редагування, повторне використання та подальше розширення програмного продукту.

У системі предметів одиниця даних відокремлена від одиниці поведінки. `ItemScriptableObject` зберігає статичні властивості предметів, тоді як компоненти сцени використовують ці дані для виконання певних дій, таких як відображення предмета в інвентарі або застосування ефекту споживання. Це означає, що зміна опису предмета не вимагає переписування логіки взаємодії, оскільки дані змінюються на рівні ресурсів, а поведінка залишається окремою в межах компонентів `MonoBehaviour`. Цей принцип зменшує взаємозалежність між частинами системи та спрощує керування її архітектурою.

Основні напрями взаємодії компонентів можна подати так:

1. `ItemScriptableObject` зберігає статичні дані предмета.
2. Префаб предмета представляє його фізичний або сценовий екземпляр.
3. Компонент інвентаря отримує посилання на дані предмета.
4. UI-компоненти використовують назву, іконку та опис для відображення інформації.
5. Логіка споживання звертається до параметрів `healthChange`, `hungerChange` та `thirstChange`.

Цей список показує, що дані про предмети використовуються кількома частинами системи одночасно. Тому має сенс перенести ці дані до `ScriptableObject`: цей ресурс стає спільною точкою доступу для інвентарю, інтерфейсу та логіки використання предметів. Це дозволяє компонентам працювати з однією й тією ж інформацією без дублювання полів у кожному окремому скрипті.

Обмін даними між модулями здійснюється через посилання на об'єкти та виклики функцій. Статичні властивості предметів передаються через `ItemScriptableObject`, тоді як відповідні компоненти управління повинні обробляти динамічні стани системи, такі як поточна кількість предметів або зміни вказівника символів. У цій моделі `ScriptableObject` не замінює менеджер інвентарю; він просто надає йому дані, необхідні для виконання операцій. Це дозволяє відокремити описи інформації про предмети від поточного стану гри.

Документація та організація коду підтримують чіткі назви полів, логічне групування даних та використання атрибутів Unity. Елемент [Header] допомагає розділити набори параметрів у вікні Інспектора, тоді як елемент [TextArea] спрощує редагування текстових описів для об'єктів. Ці інструменти не змінюють логіку виконання програми, але покращують управління ресурсами в редакторі та зменшують ймовірність помилок конфігурації. В результаті, структура даних стає зрозумілою не лише програмісту, але й користувачеві редактора Unity.

3.5.4 Інтерфейс користувача та збереження даних

Користувацький інтерфейс проекту пов'язаний із системою предметів, надаючи користувачеві інформацію про назву, значок, опис та тип предмета. Дані відображення отримуються з об'єкта `ItemScriptableObject`, що дозволяє використовувати один і той самий ресурс як для логіки інвентаризації, так і

для візуального представлення. Такий підхід забезпечує узгодженість між внутрішніми даними системи та тим, що користувач бачить на екрані. Якщо предмет змінюється на рівні ресурсу, усі компоненти, що посилаються на цей ресурс, можуть використовувати оновлену інформацію.

В оригінальному тексті зазначено, що елементи інтерфейсу користувача, включаючи меню та панелі інвентаризації, розміщуються на об'єкті Canvas. Canvas в Unity служить основою для розміщення елементів інтерфейсу користувача, а властивість Screen Space Overlay дозволяє відображати інтерфейс користувача поверх сцени. Це має вирішальне значення в контексті системи інвентаризації, оскільки користувач повинен бачити інформацію про предмети незалежно від положення ігрової камери або розташування предмета в сцені. У цьому випадку конкретна реалізація таких елементів, як ScrollView, Button або інших компонентів інтерфейсу користувача, має бути підтверджена об'єктами сцени, портом рендерингу або кодом.

Взаємодія інтерфейсу з даними предметів може бути описана через послідовність обміну інформацією:

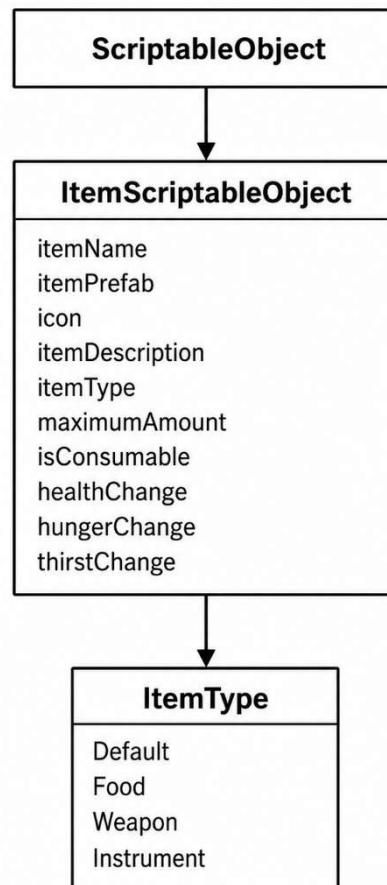


Рисунок 5 – UML-подібна модель класу `ItemScriptableObject` та переліку `ItemType`

Наведена нижче діаграма ілюструє загальний потік даних від ресурсу елемента до інтерфейсу користувача. У цій структурі `ItemScriptableObject` не відповідає за фактичне відображення, а радше надає інформацію, яку компоненти інтерфейсу користувача використовують для побудови візуального представлення. Такий поділ забезпечує логічне розмежування між

обробкою даних, управлінням запасами та відображенням результату на екрані.

Для збереження стану в тексті передбачено використання двох підходів: PlayerPrefs і JSON-серіалізації. PlayerPrefs є вбудованим механізмом Unity для збереження простих значень між ігровими сесіями, зокрема рядків, цілих чисел і чисел з плаваючою комою [15]. Оскільки PlayerPrefs не забезпечує шифрування, його доцільно застосовувати лише для некритичних параметрів або простих налаштувань. Це обмеження важливо враховувати під час опису системи збереження, щоб не приписувати їй властивостей захищеного сховища.

Для складніших структур у тексті зазначено використання JSON-формату. JsonUtility дає змогу перетворювати серіалізовані об'єкти в JSON-представлення та відновлювати їх із текстового формату [16]. У контексті інвентаря та стану предметів такий підхід може використовуватися для збереження структурованих даних, які не зручно представляти як окремі прості значення PlayerPrefs. При цьому коректність такого опису залежить від наявності відповідного коду серіалізації або файлів збереження у матеріалах проєкту.

3.5.5 Забезпечення надійності та тестування

Надійність будь-якого програмного продукту залежить від того, наскільки його окремі компоненти реагують на зміни стану, помилки введення, відсутні посилання та повторне використання даних. Відокремлення об'єктних даних від функціональної логіки має вирішальне значення в проєктуванні, оскільки така структура зменшує частоту дублювання або зміни тих самих значень. Коли об'єктні дані зберігаються в одному ресурсі ScriptableObject, інші модулі отримують узгоджену інформацію, що покращує зручність обслуговування системи. Це не усуває потреби в комплексному тестуванні, але забезпечує більш контрольовану основу для перевірки функціональності компонентів.

У вихідному тексті зазначено використання автоматизованих тестів для перевірки стабільності реалізації. Unity Test Framework підтримує тестування в Edit Mode і Play Mode, що дозволяє перевіряти як редакторну логіку, так і поведінку під час запуску гри [19], [20]. Для системи інвентаря доцільними є перевірки додавання предметів, видалення предметів, обмеження максимальної кількості в стеку, а також коректності збереження та завантаження даних. Такі перевірки допомагають виявляти помилки під час розширення функціональності.

Основні напрями тестування системи можна подати так:

- перевірка створення та використання даних ItemScriptableObject;
- перевірка правильності класифікації предметів через ItemType;
- перевірка обмеження кількості предметів відповідно до maximumAmount;
- перевірка застосування параметрів споживання для isConsumable;

- перевірка відображення назви, іконки та опису предмета в інтерфейсі;
- перевірка збереження простих і складних даних через обрані механізми.

Після проведення цих випробувань можна зробити логічні висновки щодо цілісності різних компонентів системи. Однак дані щодо фактичних результатів випробувань, кількості пройдених випробувань або відсутності дефектів у системі повинні базуватися на конкретних звітах про випробування, записах або таблицях перевірок. Якщо такі матеріали недоступні, доцільно описати лише методологію випробувань, що використовувалася або застосовувалася в дослідженні, без узагальнення результатів.

Додатковим чинником надійності є логування критичних подій і перевірка помилкових станів. У Unity для фіксації помилок може використовуватися `Debug.LogError`, який виводить повідомлення про помилку в консоль редактора [14]. Таке логування допомагає виявляти некоректні налаштування компонентів, відсутні посилання або інші ситуації, які можуть порушити роботу системи. У поєднанні з тестуванням це створює основу для більш контрольованої розробки та подальшого супроводу програмного продукту.

3.6 Реалізація модуля інвентаря

3.6.1 Призначення модуля інвентаря

Модуль інвентаря у програмному продукті призначений для відображення предметів, які можуть бути отримані, накопичені або використані користувачем у межах ігрової сцени. Його реалізація ґрунтується на використанні об'єктів Unity, компонентів користувацького інтерфейсу та C#-скриптів, які відповідають за збереження стану окремого слота інвентаря. У межах наданого фрагмента коду основним елементом цього модуля є клас `InventorySlot`, який описує логіку одного осередку інвентаря та забезпечує зв'язок між даними предмета і його візуальним представленням. Такий підхід відповідає компонентній моделі Unity, у якій поведінка об'єктів реалізується через окремі компоненти, прикріплені до `GameObject` [4], [5].

Клас `InventorySlot` виступає посередником між даними класу та елементами інтерфейсу користувача. Дані класу зберігаються у змінній `item`, яка має тип `ItemScriptableObject`, а кількість елементів у слоті визначається змінною `amount`. Стан слота контролюється логічною змінною `isEmpty`, яка розрізняє порожній слот від того, що містить елемент. Це дозволяє іншим компонентам системи перевіряти наявність слотів, можливість розміщення ідентичних елементів в одній комірці та чи потрібно оновлювати інтерфейс після зміни кількості елементів.

Основні завдання компонента `InventorySlot` полягають у такому:

- збереження посилання на предмет, який міститься у слоті;
- відображення іконки предмета в інтерфейсі користувача;
- відображення кількості предметів у слоті;
- перевірка можливості складання однакових предметів;

- очищення слота після видалення предмета;
- оновлення стану інтерфейсу після зміни кількості предметів.

Описані вище завдання демонструють, що `InventorySlot` не є комплексною системою управління запасами, а радше відповідає за стан та відображення окремого слота інвентарю. Це забезпечує чіткий розподіл обов'язків: слот зберігає дані локально та оновлює свій інтерфейс користувача, тоді як окремий компонент обробляє загальну логіку додавання товарів, пошуку доступних слотів та управління всією панеллю інструментів інвентарю. Така структура робить систему більш прозорою та придатною для майбутнього розширення.

3.6.2 Реалізація класу `InventorySlot`

Наведений клас успадковується від `MonoBehaviour`, тому може використовуватися як компонент Unity і прикріплюватися до об'єктів інтерфейсу. У методі `Awake()` виконується початкова ініціалізація слота, зокрема отримання дочірніх об'єктів, які відповідають за іконку предмета та текстове поле кількості. Перед зверненням до дочірніх елементів реалізовано перевірку `transform.childCount < 2`, яка запобігає некоректному доступу до елементів ієрархії у випадку неправильно налаштованої структури UI-об'єкта. Якщо потрібні дочірні об'єкти відсутні, у консоль Unity виводиться повідомлення через `Debug.LogError`, що спрощує виявлення помилок під час налаштування сцени [14].

Після перевірки структури об'єкта, функція `Awake()` отримує посилання на перший дочірній об'єкт як `icon`, а від другого дочірнього об'єкта отримує компонент `TMP_Text`. Далі викликається функція `ClearSlot()`, яка повертає слот до його початкового порожнього стану. Ця послідовність ініціалізації гарантує, що слот готовий до подальших операцій, перш ніж будуть додані будь-які елементи. В результаті, елемент інтерфейсу не має залишкових випадкових даних і одразу отримує узгоджений початковий стан.

Функція `SetItem()` використовується для заповнення слота новим елементом та визначення його кількості. Ця функція призначає посилання на об'єкт `ItemScriptableObject` змінній `item`, кількість елементів – змінній `amount`, а значення `false` – змінній `isEmpty`. Після оновлення внутрішнього стану викликаються функції `SetIcon()` та `UpdateAmount()`, тобто вони негайно синхронізують дані слота з його візуальним представленням. Таке розділення дозволяє відокремити логіку призначення елементів від логіки оновлення значка та текстового поля.

Функція `SetIcon()` відповідає за встановлення графічного зображення елемента в інтерфейсі користувача. Ця функція бере компонент зображення з об'єкта `iconGO` та призначає йому передане зображення. Крім того, зображення встановлюється на білий колір, що робить значок видимим після заповнення поля. Коли поле очищено, той самий об'єкт використовується для приховування значка шляхом видалення зображення та встановлення для нього прозорого кольору.

3.6.3 Логіка зміни кількості предметів

Зміна кількості предметів у слоті реалізована через методи `AddAmount()` і `RemoveAmount()`. Метод `AddAmount()` збільшує значення змінної `amount` на передане число та одразу викликає `UpdateAmount()`, щоб відобразити зміну в інтерфейсі. Таким чином, кожна зміна числового значення супроводжується оновленням UI, що забезпечує узгодженість між внутрішнім станом слота та видимою інформацією для користувача. Це важливо для інвентаря, оскільки користувач повинен бачити актуальну кількість предметів після кожної операції додавання.

Метод `RemoveAmount()` виконує протилежну дію: він зменшує кількість предметів у слоті на передане значення. Після цього метод перевіряє, чи залишилися предмети в комірці. Якщо значення `amount` стає меншим або дорівнює нулю, викликається `ClearSlot()`, що повертає слот у початковий порожній стан. Якщо предмети ще залишаються, виконується лише оновлення текстового поля кількості через `UpdateAmount()`.

Окрему роль у логіці інвентаря виконує метод `CanStackWith()`. Він перевіряє, чи можна додати новий предмет до вже зайнятого слота, тобто чи слот не є порожнім, чи предмет збігається з поточним, а також чи кількість предметів не перевищує максимально допустиме значення. Максимальна кількість береться з поля `maxAmount` об'єкта `ItemScriptableObject`; назву цього поля необхідно узгодити з фактичною реалізацією класу предмета, оскільки в ній може бути орфографічна неточність. За логікою роботи методу, складання можливе лише для однакових предметів і лише тоді, коли в слоті ще є вільний простір.

Логіку роботи зі стеком предметів можна подати у вигляді схеми:

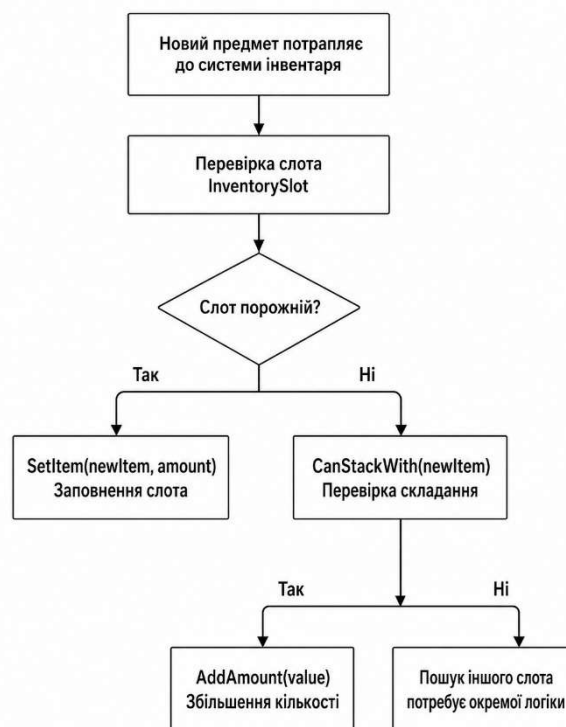


Рисунок 6 – Алгоритм додавання предмета до інвентарного слота

Схема показує, що InventorySlot виконує локальну перевірку стану одного слота, але не відповідає за повний пошук місця в інвентарі. Якщо слот порожній, він може бути заповнений через SetItem(), а якщо слот зайнятий, спочатку перевіряється можливість складання предметів через CanStackWith(). У разі неможливості складання потрібна логіка пошуку іншого слота, яка має бути реалізована в окремому компоненті керування інвентарем.

Метод GetFreeSpace() повертає кількість вільних місць у поточному слоті. Якщо слот порожній або предмет не призначений, метод повертає нуль, що запобігає некоректним обчисленням. Якщо слот містить предмет, вільний простір визначається як різниця між максимальною кількістю предметів і поточним значенням amount. Така реалізація дає змогу іншим частинам системи визначати, скільки одиниць можна додати до конкретної комірки без перевищення встановленого ліміту.

3.6.4 Взаємодія компонента з інтерфейсом користувача

Користувацький інтерфейс у цьому фрагменті реалізується через поєднання компонента Image та текстового компонента TMP_Text. Іконка предмета відображається через об'єкт iconGO, а кількість предметів показується в полі ItemAmountText. Якщо кількість предметів дорівнює одному, текстове поле залишається порожнім, оскільки одиничний предмет не потребує окремого числового позначення. Якщо предметів більше одного, у слоті відображається відповідне числове значення.

Структура UI-об'єкта слота повинна відповідати логіці, закладеній у методі Awake(). Перший дочірній об'єкт має бути призначений для іконки, а другий — для тексту кількості. [Якщо ця структура буде порушена, компонент не зможе коректно отримати посилання на необхідні елементи інтерфейсу.] Тому під час налаштування сцени необхідно перевірити ієрархію об'єкта слота в Unity Hierarchy та Inspector.

Взаємодія між даними предмета, слотом інвентаря та UI може бути подана так:

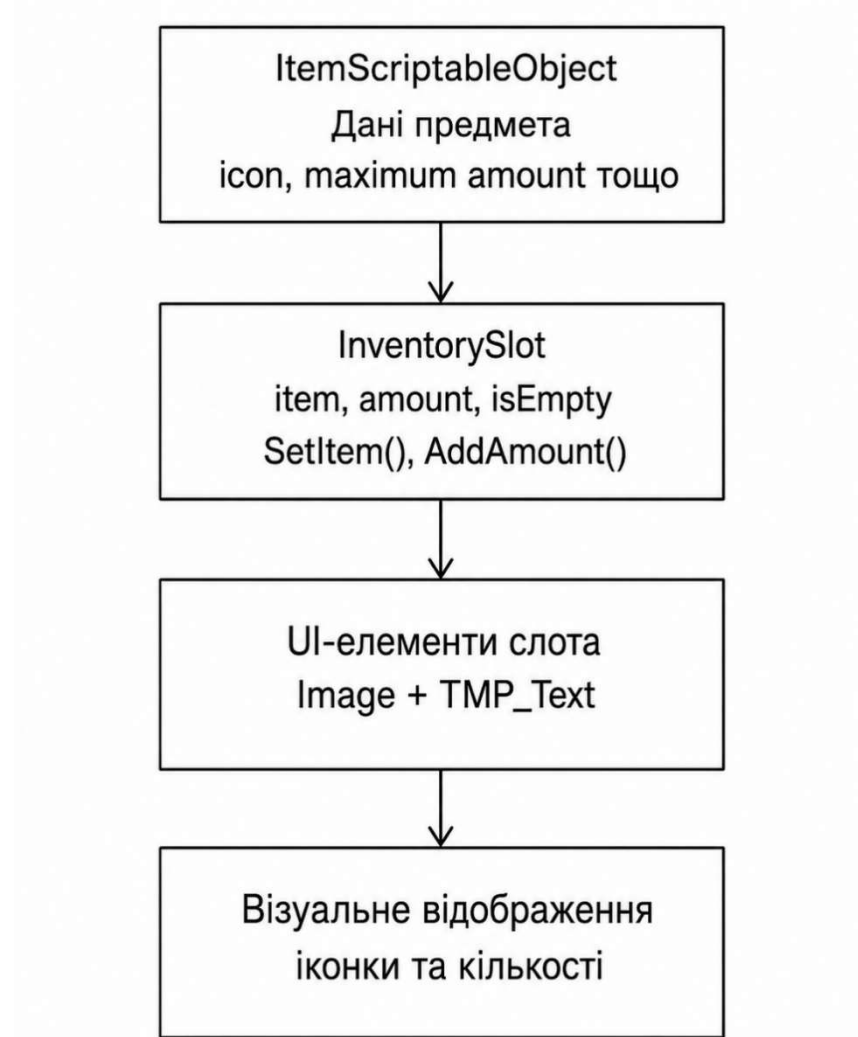


Рисунок 7 – Потік даних від ItemScriptableObject до візуального представлення слота

Наведена схема демонструє, що ItemScriptableObject є джерелом даних про предмет, а InventorySlot використовує ці дані для оновлення елементів інтерфейсу. Компонент не створює дані предмета самостійно, а отримує їх через посилання, після чого відображає іконку та кількість у відповідних UI-елементах. Такий підхід забезпечує логічне розділення між ресурсами предметів, станом слота та візуальним шаром інтерфейсу.

3.6.5 Обробка помилок і стабільність роботи

У класі InventorySlot передбачено базову обробку помилок, пов'язану з неправильною структурою UI-об'єкта. Перевірка кількості дочірніх елементів у методі Awake() дає змогу виявити ситуацію, коли слот не має необхідних об'єктів для відображення іконки та кількості. [У такому випадку в консоль Unity виводиться повідомлення про помилку із зазначенням назви об'єкта, що

спрощує пошук проблемного елемента сцени.] Це зменшує ризик непомітних помилок під час налаштування інтерфейсу.

Додаткові перевірки реалізовано в методах `UpdateAmount()` і `ClearSlot()`. У методі `UpdateAmount()` перевіряється, чи не дорівнює `ItemAmountText` значенню `null`, що запобігає зверненню до неініціалізованого текстового компонента. У методі `ClearSlot()` перед зміною параметрів перевіряється наявність `iconGO` і `ItemAmountText`, тому компонент може безпечніше працювати навіть за частково некоректного налаштування UI. Такий підхід підвищує стабільність роботи окремого слота інвентаря.

Для підвищення надійності компонента доцільно перевірити такі аспекти:

1. Коректність структури UI-об'єкта:
 - наявність дочірнього об'єкта для іконки;
 - наявність дочірнього об'єкта для тексту;
 - наявність компонента `Image` на об'єкті іконки;
 - наявність компонента `TMP_Text` на текстовому об'єкті.
2. Коректність даних предмета:
 - наявність іконки в `ItemScriptableObject`;
 - правильність максимального значення кількості;
 - відповідність назви поля `maxmumAmont` фактичній реалізації класу предмета;
 - відсутність порожніх посилань на об'єкти даних.

Наведені перевірки стосуються як структури інтерфейсу, так і даних, які використовує слот. Їх виконання дає змогу зменшити кількість помилок, пов'язаних із неправильним налаштуванням `Inspector`, відсутністю компонентів або невідповідністю структури даних предмета. Завдяки цьому компонент `InventorySlot` може стабільніше працювати в складі загальної системи інвентаря.

3.6.6 Використання принципів об'єктно-орієнтованого програмування

Реалізація класу `InventorySlot` демонструє використання базових принципів об'єктно-орієнтованого програмування. Клас інкапсулює стан одного слота інвентаря та методи, які змінюють цей стан. Замість прямого керування всіма полями з інших компонентів передбачено окремі методи для встановлення предмета, додавання кількості, зменшення кількості та очищення слота. Це робить компонент зрозумілішим і зменшує ймовірність неконтрольованої зміни його стану.

Модульність реалізації полягає в тому, що `InventorySlot` відповідає лише за один слот, а не за всю систему інвентаря. Загальна логіка додавання предметів, пошуку вільних слотів або збереження стану інвентаря має бути реалізована в окремому компоненті керування інвентарем. Такий розподіл відповідальності дає змогу уникнути надмірного ускладнення одного класу та полегшує подальше розширення функціональності. Крім того, окремий клас слота можна повторно використовувати для різних панелей інвентаря, якщо їхня UI-структура залишається однаковою.

Методи класу мають чітко визначені ролі. `SetItem()` встановлює предмет і кількість, `SetIcon()` оновлює графічне представлення, `UpdateAmount()` синхронізує текст кількості, `AddAmount()` і `RemoveAmount()` змінюють числовий стан, а `ClearSlot()` повертає слот у порожній стан. [Завдяки такому поділу логіка класу залишається послідовною: кожен метод виконує окрему дію, а складніша поведінка формується через їх поєднання.] Це відповідає принципу єдиної відповідальності на рівні окремого компонента інтерфейсу.

3.6.7 Збереження даних, документування та тестування

У наведеному класі `InventorySlot` збереження даних інвентаря у файл, базу даних або систему `PlayerPrefs` не реалізовано. Компонент зберігає стан лише під час виконання сцени, тобто інформація про предмет і його кількість існує в оперативному стані об'єкта `Unity`. [Для повноцінного збереження прогресу необхідно реалізувати окремий модуль, який буде збирати дані зі слотів інвентаря та записувати їх у вибране сховище.] Якщо в проєкті передбачено збереження інвентаря після завершення гри, цей механізм потрібно описувати окремо на основі відповідного коду.

Документування коду частково забезпечується зрозумілими назвами методів, які відображають їхнє призначення. Наприклад, назви `SetItem()`, `UpdateAmount()`, `AddAmount()`, `RemoveAmount()` і `ClearSlot()` дають змогу швидко визначити, яку дію виконує кожний метод. Водночас для методів, які використовуються іншими компонентами системи, доцільно додати короткі коментарі або XML-документацію. Такий підхід полегшить подальше розширення інвентаря та зменшить ризик неправильного використання методів [18].

Перевірка функціоналу модуля інвентаря повинна охоплювати як логіку даних, так і коректність UI-відображення. Насамперед потрібно перевірити створення порожнього слота під час запуску сцени, додавання нового предмета, оновлення кількості, складання однакових предметів і очищення слота після видалення останнього предмета. Окремо слід перевірити ситуації, коли в об'єкті слота відсутні дочірні UI-елементи або не призначено іконку предмета. Таке тестування дасть змогу оцінити стабільність роботи компонента в типових і помилкових сценаріях.

Рекомендовані сценарії тестування модуля:

- запуск сцени з порожніми слотами інвентаря;
- додавання предмета в порожній слот;
- додавання кількох однакових предметів;
- перевірка обмеження максимальної кількості предметів;
- зменшення кількості предметів у слоті;
- очищення слота після видалення останнього предмета;
- перевірка відображення іконки та кількості;
- перевірка повідомлення про помилку за неправильної структури UI-об'єкта.

Наведені сценарії охоплюють основні операції, які виконує InventorySlot, і дають змогу перевірити зв'язок між внутрішнім станом слота та його відображенням у UI. Якщо тестування виконується через Unity Test Framework, можна окремо розділити перевірки логіки компонента та перевірки поведінки під час запуску сцени. Фактичні результати тестування необхідно подавати лише за наявності тестових звітів, логів або таблиць перевірки [19], [20].

3.7 Реалізація системи інвентаря та інтерактивних об'єктів

3.7.1 Загальна характеристика реалізованих модулів

У межах реалізації програмного продукту на Unity система інвентаря побудована як сукупність окремих компонентів, кожен із яких відповідає за визначену частину функціональності. Основними елементами цієї системи є менеджер інвентаря, слоти інвентаря, об'єкти предметів, механізм перетягування елементів інтерфейсу та інтерактивні об'єкти сцени. Такий підхід відповідає компонентній моделі Unity, у якій поведінка ігрових об'єктів реалізується через C#-скрипти, що прикріплюються до GameObject як компоненти. Розділення логіки між окремими класами робить систему зрозумілішою для подальшого розширення, тестування та супроводу [4], [5].

Реалізація інвентаря передбачає взаємодію між користувацьким інтерфейсом, даними предметів і логікою додавання об'єктів до слотів. Компонент InventoryManager відповідає за відкривання та закривання інвентаря, збирання слотів із UI-панелі та додавання предметів. Клас InventorySlot зберігає інформацію про конкретний предмет, його кількість і стан окремої комірки. Додатково використовується клас DragAndDropItem, який забезпечує переміщення предметів між слотами за допомогою подій інтерфейсу.

Основні складові реалізованої системи можна подати таким чином:

- модуль керування інвентарем:
 - відкривання та закривання UI;
 - ініціалізація слотів;
 - додавання предметів;
 - перевірка переповнення інвентаря;
- модуль слота інвентаря:
 - збереження предмета;
 - відображення іконки;
 - оновлення кількості;
 - очищення слота;
- модуль взаємодії з UI:
 - перетягування предметів;
 - переміщення між слотами;
 - обмін предметами;
- модуль інтерактивних об'єктів:
 - відкривання дверей;

- висування ящика або коробки;
- використання спільної базової логіки `OpenableObject`.

Наведена структура показує, що система не є монолітною, а складається з кількох взаємопов'язаних частин. Кожен компонент виконує окрему роль: менеджер керує загальним станом інвентаря, слот відповідає за одну комірку, компонент перетягування забезпечує зміну розташування предметів, а класи інтерактивних об'єктів реалізують поведінку елементів сцени. Такий розподіл відповідальності полегшує аналіз коду та дає змогу доопрацьовувати окремі частини системи без повного переписування інших модулів.

3.7.2 Реалізація менеджера інвентаря

Клас `InventoryManager` є центральним компонентом системи інвентаря, оскільки він координує роботу UI-панелі, списку слотів і логіки додавання предметів. Він успадковується від `MonoBehaviour`, тому може бути прикріплений до об'єкта сцени в Unity та використовувати методи життєвого циклу `Awake()`, `Start()` і `Update()` [5]. У методі `Awake()` встановлюється початковий стан інвентаря, а також приховуються фоновий UI-об'єкт `UIBG` і панель `InventoryPanel`. Повторне приховування цих елементів у методі `Start()` підвищує стабільність запуску сцени, оскільки гарантує закритий стан інвентаря після ініціалізації слотів.

Важливою частиною реалізації є статична властивість `IsInventoryOpen`, яка зберігає загальний стан відкриття інвентаря. Завдяки цьому інші компоненти можуть перевіряти, чи відкрито інвентар, і змінювати свою поведінку відповідно до цього стану. Наприклад, система взаємодії з предметами або об'єктами сцени може тимчасово блокувати обробку дій користувача, якщо інвентар відкритий. Такий підхід забезпечує узгодженість між користувацьким інтерфейсом і керуванням персонажем.

Метод `InitializeSlots()` виконує автоматичне збирання слотів інвентаря з дочірніх елементів `InventoryPanel`. Це рішення дає змогу не додавати кожен слот вручну до списку `slots`, а формувати його на основі фактичної структури UI-панелі. Якщо `InventoryPanel` не призначено в Inspector, у консоль Unity виводиться повідомлення про помилку через `Debug.LogError`, що допомагає виявити неправильне налаштування компонента. Така перевірка є елементом базової обробки помилок і спрощує налагодження сцени [14].

Логіка відкривання та закривання інвентаря реалізована в методі `ToggleInventory()`. Після натискання клавіші `Tab` значення `isOpened` змінюється на протилежне, а властивість `IsInventoryOpen` синхронізується з цим станом. Далі активуються або деактивуються UI-елементи інвентаря, а також змінюється стан курсора: під час відкритого інвентаря курсор стає видимим і розблокованим, а після закриття знову блокується. Обробка натискання клавіші виконується через `Input.GetKeyDown`, який повертає істину в кадрі, коли користувач починає натискати відповідну клавішу [12].

3.7.3 Алгоритм додавання предметів до інвентаря

Метод `AddItem()` реалізує основну логіку додавання предметів до інвентаря. Спочатку виконується перевірка коректності вхідних даних: предмет не повинен дорівнювати `null`, а кількість має бути більшою за нуль. Якщо ці умови не виконуються, метод припиняє роботу та виводить повідомлення про помилку або попередження. Це запобігає некоректному оновленню слотів і підвищує стабільність системи.

Алгоритм додавання предметів складається з двох основних етапів. На першому етапі система перевіряє вже зайняті слоти та намагається додати предмет до наявного стека, якщо предмети однакові та максимальна кількість у слоті ще не досягнута. На другому етапі, якщо частина предметів залишилася, система шукає порожній слот і розміщує предмет у ньому. Якщо після проходження всіх слотів залишається невміщена кількість предметів, у консоль виводиться попередження про переповнення інвентаря.

Логіку додавання предмета до інвентаря можна подати у вигляді блок-схеми:

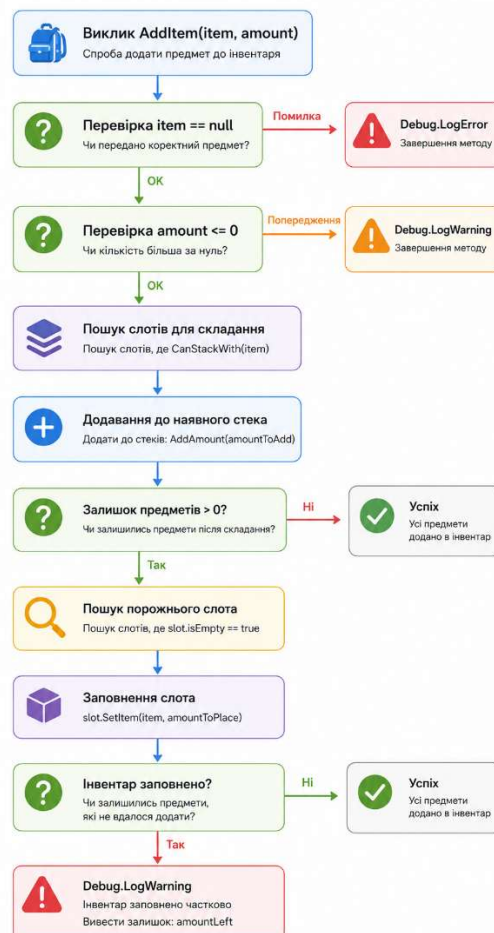


Рисунок 8 – Блок-схема алгоритму додавання предметів до інвентаря

Наведена блок-схема відображає послідовність перевірок і дій, які виконуються під час додавання предмета. Спочатку система відсікає некоректні вхідні дані, після чого намагається використати вже наявні стеки, а лише потім переходить до пошуку порожнього слота. Такий порядок є

логічним, оскільки він дає змогу раціонально використовувати простір інвентаря та не створювати зайві заповнені комірки для однакових предметів.

Особливістю реалізації є використання змінної `remainingAmount`, яка зберігає кількість предметів, що ще не були розміщені в інвентарі. Це дає змогу коректно обробляти випадки, коли предметів більше, ніж може вмістити один слот. Значення `amountToAdd` і `amountToPlace` обчислюються через `Mathf.Min`, що запобігає перевищенню максимально допустимої кількості предметів у слоті. Максимальна кількість береться з поля `item.maxmumAmont`, тому назву цього поля необхідно узгодити з фактичною реалізацією класу `ItemScriptableObject`.

3.7.4 Реалізація типу предмета `FoodItem`

Клас `FoodItem` є спеціалізованим типом предмета, який успадковується від базового класу `ItemScriptableObject`. Його призначення полягає у створенні предметів типу їжі, які можуть мати додатковий параметр `healAmount`. [Це поле вказує на числовий ефект, пов'язаний із використанням предмета, наприклад відновлення певного показника. Оскільки повний код базового класу `ItemScriptableObject` у цьому фрагменті не наведено, детальний опис усіх успадкованих полів потребує звірення з фактичною реалізацією.

Атрибут `CreateAssetMenu` дає змогу створювати об'єкти цього типу безпосередньо через меню `Unity Editor`. Це спрощує наповнення системи предметів, оскільки розробник може створювати окремі `asset`-файли для різних видів їжі. Такі об'єкти можуть містити назву, іконку, максимальну кількість у слоті та інші параметри, якщо вони передбачені в базовому класі `ItemScriptableObject`. У межах системи інвентаря `FoodItem` використовується як джерело даних, а не як сценовий об'єкт.

У методі `Start()` значення `itemType` встановлюється як `ItemType.Food`. Проте для класу, що успадковується від `ScriptableObject`, використання такого способу ініціалізації потребує перевірки, оскільки `ScriptableObject` є серіалізованим типом `Unity`, який зазвичай створюється як окремий ресурс проєкту [6]. Для більш контрольованої реалізації тип предмета доцільно задавати під час створення відповідного ресурсу або через спеціально визначену логіку ініціалізації. Це допоможе уникнути ситуацій, коли тип предмета не встановлюється очікуваним способом.

3.7.5 Реалізація перетягування предметів між слотами

Клас `DragAndDropItem` реалізує механізм перетягування предметів між слотами інвентаря. Для цього він використовує інтерфейси `IBeginDragHandler`, `IDragHandler` та `IEndDragHandler`, які обробляють початок перетягування, сам процес переміщення та завершення дії. На початку перетягування компонент визначає слот, з якого було взято предмет, і перевіряє, чи він не є порожнім. Якщо слот порожній або не знайдений, подальші дії не виконуються, що запобігає некоректному перенесенню порожнього елемента.

Під час початку перетягування зберігаються початковий батьківський об'єкт і позиція елемента. Потім об'єкт тимчасово переноситься до Canvas і встановлюється останнім дочірнім елементом, щоб відображатися поверх інших UI-компонентів. Під час руху миші позиція елемента оновлюється відповідно до `Input.mousePosition`, завдяки чому об'єкт візуально рухається за курсором. Після завершення перетягування система визначає слот, над яким знаходиться курсор, і виконує переміщення або обмін предметами.

Метод `MoveOrSwap()` містить дві основні гілки логіки. Якщо цільовий слот порожній, предмет переноситься до нього, а початковий слот очищується. Якщо цільовий слот уже містить інший предмет, виконується обмін даними між двома слотами. Для цього тимчасово зберігаються предмет і кількість із цільового слота, після чого значення слотів міняються місцями.

Схема взаємодії між компонентами під час перетягування предмета має такий вигляд:

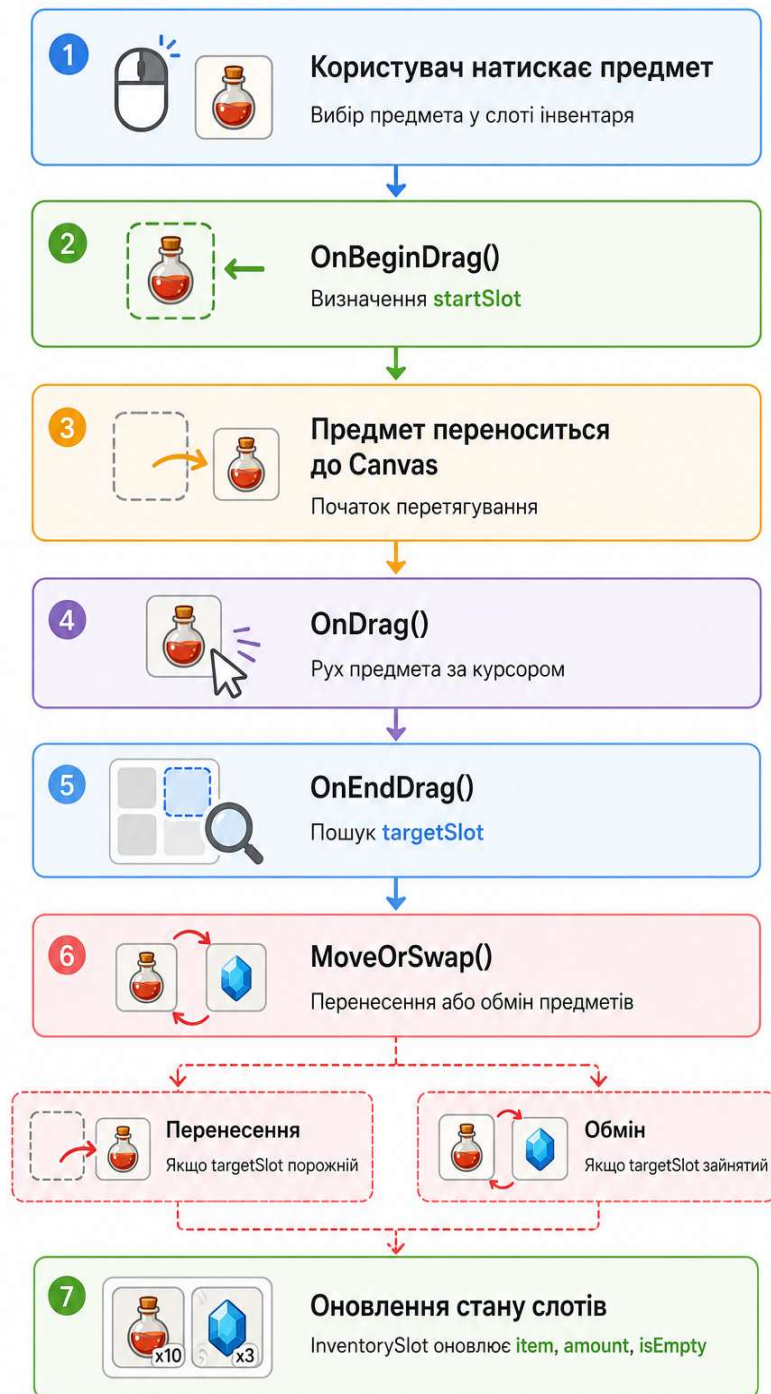


Рисунок 9 – Послідовність роботи компонентів під час виконання операції Drag-and-Drop

Наведена схема показує послідовність роботи механізму перетягування від моменту натискання на предмет до оновлення стану слотів. Клас `DragAndDropItem` не зберігає предмет як самостійну одиницю інвентаря, а працює з даними через компоненти `InventorySlot`. Завдяки цьому переміщення в інтерфейсі приводить до зміни логічного стану відповідних слотів.

3.7.6 Реалізація інтерактивних об'єктів сцени

Інтерактивні об'єкти сцени представлені класами Door і Box, які успадковуються від базового класу OpenableObject. Така структура демонструє використання наслідування як одного з принципів об'єктно-орієнтованого програмування. Спільна логіка відкривання та закривання винесена до базового класу, а конкретні дочірні класи реалізують власний спосіб зміни стану об'єкта. Оскільки код OpenableObject у наведеному фрагменті не подано, його поля та методи необхідно описувати окремо після аналізу фактичної реалізації.

Клас Door реалізує відкривання та закривання дверей шляхом плавної зміни їхнього обертання. У методі Start() зберігається початковий кут повороту об'єкта, після чого на основі значення _directions обчислюється кінцева орієнтація. Для визначення напрямку використовується перелік Directions, який у цій логіці має містити значення Forward, Right і Up. Значення _rotateByDegrees визначає, на скільки градусів двері мають повернутися під час відкривання.

Методи Open() і Close() реалізовані як корутини, що дає змогу виконувати плавний рух протягом кількох кадрів. Для інтерполяції між початковим і кінцевим станами використовується Quaternion.Lerp, а значення _openOrCloseLerp поступово змінюється з урахуванням Time.deltaTime. Застосування Mathf.Clamp01 обмежує значення інтерполяції в межах від 0 до 1, що запобігає виходу за допустимий діапазон. Змінна _isMoving використовується для блокування повторного запуску відкривання або закривання під час уже активної анімації.

Клас Box реалізує інший тип відкритого об'єкта, який змінює не обертання, а позицію. У методі Start() зберігається початкова позиція об'єкта, після чого визначається кінцева позиція з урахуванням напрямку руху. Поле _moveTolenght задає довжину зміщення об'єкта, тобто відстань, на яку коробка або ящик висувається під час відкривання. У назві цього поля бажано перевірити орфографію, оскільки для читабельності коду доцільніше використовувати назву на зразок _moveLength.

Методи Open() і Close() у класі Box мають таку саму загальну структуру, як і в класі Door, однак замість Quaternion.Lerp використовується Vector3.Lerp. Це свідчить про повторне використання загального архітектурного підходу для різних типів інтерактивних об'єктів. Двері відкриваються через зміну повороту, а коробка або ящик — через зміну позиції. Така реалізація демонструє поліморфізм, оскільки обидва класи можуть викликатися через спільний тип OpenableObject, але виконувати різну поведінку.

3.7.7 Взаємозв'язок між системою інвентаря та інтерактивними об'єктами

Система інвентаря та система інтерактивних об'єктів є окремими модулями, однак вони можуть працювати в межах єдиної логіки взаємодії користувача зі сценою. Інвентар відповідає за збереження та відображення предметів, тоді як класи Door і Box відповідають за зміну стану об'єктів

середовища. Зв'язок між ними може здійснюватися через окремий компонент взаємодії, який визначає, на який об'єкт спрямований погляд або курсор користувача. Такий компонент може викликати відкривання дверей, висування коробки або додавання предмета до інвентаря, якщо відповідна логіка передбачена в проєкті.

Загальну архітектурну взаємодію компонентів можна подати так:

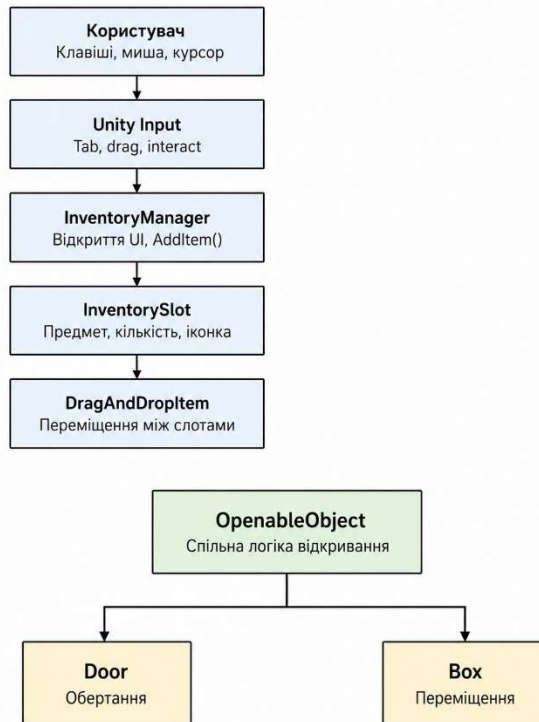


Рисунок 10 – Взаємозв'язок компонентів інвентарю та інтерактивних об'єктів сцени

Наведена схема демонструє розділення системи на два основні напрями: керування інвентарем і взаємодію з відкривними об'єктами сцени. Перший напрям охоплює відкривання UI, додавання предметів, роботу зі слотами та перетягування елементів, а другий — зміну стану об'єктів через спільну базову логіку `OpenableObject`. Така схема допомагає показати, що модулі можуть бути пов'язані загальною системою введення, але не повинні дублювати відповідальність один одного.

З погляду модульності, кожен клас має окрему відповідальність. `InventoryManager` не керує безпосередньо відкриванням дверей, а `Door` і `Box` не містять логіки інвентаря. Такий розподіл зменшує залежність між компонентами та спрощує внесення змін до окремих частин системи. Наприклад, можна змінити механіку відкривання дверей без переписування коду інвентаря або додати новий тип предмета без зміни логіки відкривних об'єктів.

3.7.8 Обробка помилок, стабільність і тестування

У реалізованих класах передбачено базові засоби обробки помилок, які спрямовані на виявлення некоректного налаштування компонентів. У класі

InventoryManager перевіряється наявність InventoryPanel, а також коректність предмета й кількості під час виклику AddItem(). Якщо предмет не заданий, виводиться помилка через Debug.LogError; якщо кількість некоректна, виводиться попередження через Debug.LogWarning. У випадку переповнення інвентаря система також повідомляє про залишок предметів, які не вдалося розмістити.

У класах Door і Box стабільність роботи забезпечується через змінну _isMoving. Вона не дозволяє повторно запускати корутину відкривання або закривання, поки попередній рух ще не завершився. Це запобігає конфлікту між кількома одночасними змінами позиції або обертання об'єкта. Додатково значення _openOrCloseLerp обмежується методом Mathf.Clamp01, що зменшує ризик некоректного завершення анімації.

Для перевірки реалізованого функціоналу доцільно виконати такі тестові сценарії:

1. Перевірка інвентаря:
 - запуск сцени із закритим інвентарем;
 - відкривання інвентаря клавішею Tab;
 - закривання інвентаря клавішею Tab;
 - перевірка видимості курсора під час відкритого інвентаря;
 - перевірка блокування курсора після закриття інвентаря.
2. Перевірка додавання предметів:
 - додавання предмета в порожній слот;
 - додавання однакових предметів до наявного стека;
 - додавання кількості, що перевищує місткість одного слота;
 - перевірка повідомлення про переповнення інвентаря;
 - перевірка обробки null-предмета.
3. Перевірка перетягування:
 - переміщення предмета в порожній слот;
 - обмін предметами між двома зайнятими слотами;
 - спроба перетягування з порожнього слота;
 - повернення UI-елемента до початкового батьківського об'єкта.
4. Перевірка інтерактивних об'єктів:
 - відкривання дверей;
 - закривання дверей;
 - відкривання коробки або ящика;
 - закривання коробки або ящика;
 - перевірка відсутності повторного запуску руху під час активної анімації.

Наведені сценарії охоплюють основні функціональні частини системи та дають змогу перевірити зв'язок між введенням користувача, станом інвентаря, UI-елементами та інтерактивними об'єктами сцени. [Якщо тестування виконується через Unity Test Framework, можна окремо перевіряти логіку в режимах Edit Mode і Play Mode, оскільки ці режими призначені для різних типів перевірок.] Фактичні результати тестування необхідно подавати лише за наявності тестових звітів, логів або таблиць перевірки [19], [20].

3.7.9 Збереження даних і документування коду

Згадані вище фрагменти коду не зберігають стан інвентарю назавжди після завершення сцени або виходу з гри. Дані про предмети, кількість та позиція слота зберігаються лише під час виконання програми. Щоб зберегти повний прогрес, потрібен окремий модуль коду для зчитування стану всіх слотів та запису його у файл, базу даних або інший механізм зберігання. У цьому контексті слід уточнити, чи є збереження інвентарю основною функціональною вимогою.

Документація коду частково надається через чіткі назви класів, методів та полів. Наприклад, назви `AddItem()`, `ToggleInventory()`, `InitializeSlots()`, `MoveOrSwap()`, `Open()` та `Close()` відображають призначення відповідних операцій. Однак деякі фрагменти коду можуть містити орфографічні помилки, зокрема `maxmumAmont`, `_openOrCloseTime` та `_moveTolenght`. Для покращення читабельності коду та якості документації рекомендується виправити або стандартизувати ці назви.

Для підвищення якості документування доцільно додати короткі пояснення до методів, які використовуються іншими компонентами системи. У C# для цього можуть застосовуватися XML-коментарі, зокрема тег `<summary>`, який описує призначення типу або члена класу [18]. У межах цього проєкту насамперед доцільно документувати методи `AddItem()`, `SetItem()`, `MoveOrSwap()`, `Open()` і `Close()`, оскільки вони визначають ключову взаємодію між компонентами. Це полегшить подальше розширення системи та зменшить ризик неправильного використання методів у майбутніх змінах.

Таким чином, реалізована система поєднує кілька взаємопов'язаних модулів: інвентар, UI-слоти, перетягування предметів і відкриті об'єкти сцени. Менеджер інвентаря відповідає за загальне керування UI та додавання предметів, тоді як окремі слоти зберігають стан конкретних елементів. Класи `Door` і `Box` демонструють реалізацію інтерактивного середовища через наслідування від спільного базового класу. Подальше доопрацювання цього розділу потребує уточнення коду `ItemScriptableObject`, `InventorySlot`, `OpenableObject`, переліку `Directions`, механізму взаємодії користувача зі сценою та фактичного способу збереження даних.

4 ТЕСТУВАННЯ ПРОГРАМИ

4.1 Мета та об'єкти тестування

Тест мав на меті перевірити основні функції програми та забезпечити їх відповідність вимогам. Тест охоплював керування персонажами, взаємодію предметів, систему інвентарю та інтерфейс користувача. Особлива увага була приділена сценаріям, де кілька компонентів працювали разом, оскільки помилки є поширеними в таких взаємодіях.

Елементи тестування включали рух персонажа, обертання камери, стрибки, відкриття інвентарю, вибір предметів, оновлення інвентарю та реакцію програми на неправильні налаштування. Були використані функціональні, інтегративні та ручні методи тестування інтерфейсу користувача. Такий підхід дозволяє оцінити програму з точки зору користувача та перевірити узгодженість різних модулів.

4.2 Опис тестового середовища

Тестування проводилося в середовищі Unity з використанням мови програмування C#. Тест виконувався в режимі сцени, що дозволило нам оцінити фактичну поведінку персонажа, предметів та інвентарю під час виконання програми. Оскільки деякі параметри в Unity встановлюються через вікно Інспектора, тест також перевіряв правильність призначення компонентів, посилань та значень, необхідних для функціонування програми.

Тестове середовище доцільно оформити так: операційна система — Windows 10, версія Unity — 2022.3.62f3, мова програмування — C#, пристрій тестування — персональний комп'ютер. Ці дані необхідні для того, щоб результати тестування були відтворюваними та зрозумілими для перевірки.

4.3 Тестові сценарії та результати тестування

Для перевірки програмного продукту було сформовано набір тестових сценаріїв, які охоплюють основну функціональність системи. Тестування передбачало запуск сцени, перевірку керування персонажем, взаємодію з предметами, відкриття інвентаря та оновлення його стану. Також перевірялися ситуації, у яких програма повинна коректно реагувати на відсутність потрібних даних або компонентів.

Таблиця 4.1 — Результати тестування програми

№	Назва тесту	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	Запуск програми	Запуск сцени	Сцена відкривається без критичних помилок	Виконано	Успішно

2	Переміщення персонажа	Натискання клавіш руху	Персонаж рухається у відповідному напрямку	Виконано	Успішно
3	Поворот камери	Рух миші	Напрямок огляду змінюється	Виконано	Успішно
4	Стрибок персонажа	Натискання клавіші стрибка	Стрибок виконується за наявності контакту з поверхнею	Виконано	Успішно
5	Відкриття інвентаря	Команда відкриття інвентаря	Інтерфейс відкривається, керування персонажем обмежується	Виконано	Виконано
6	Підбір предмета	Взаємодія з предметом	Предмет додається до інвентаря	Виконано	Виконано
7	Оновлення слота	Додавання предмета		Виконано	
8	Підбір предмета	Взаємодія з предметом	Предмет додається до інвентаря	Виконано	Виконано
9	Оновлення слота	Додавання предмета	У слоті відображається предмет і його кількість	Виконано	Виконано
10	Відсутність даних предмета	Предмет без призначених даних	Додавання не виконується, фіксується помилка	Виконано	Виконано

Наведені результати свідчать про те, що основні функції програмного продукту працюють відповідно до очікуваної логіки. Персонаж реагує на введення користувача, інвентар відкривається та впливає на режим керування, а предмети коректно передаються до інвентарної підсистеми. Перевірка помилкових ситуацій показала, що програма не повинна виконувати некоректні дії без необхідних даних.

4.4 Виявлені помилки та способи їх усунення

Під час тестування основну увагу було приділено можливим помилкам, пов'язаним із неправильним налаштуванням компонентів у Unity. До таких помилок належать відсутність посилань на потрібні об'єкти, непризначені дані

предмета або відсутність менеджера інвентаря в сцені. Для усунення таких ситуацій у програмній логіці доцільно використовувати перевірки перед виконанням основних операцій і повідомлення про помилки в консолі.

За результатами перевірки критичних помилок, які унеможлиблюють виконання основних сценаріїв, не виявлено. Основні функції програми працюють відповідно до поставлених вимог. Подальше вдосконалення тестування може передбачати додавання автоматизованих перевірок, розширення кількості тест-кейсів та окрему перевірку роботи інтерфейсу в різних станах програми.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено програмний продукт у середовищі Unity із використанням мови програмування C#. У роботі було проаналізовано предметну область, визначено основні проблеми розробки інтерактивних систем, розглянуто підходи до організації керування персонажем, взаємодії з предметами та роботи інвентарної підсистеми. На основі проведеного аналізу було обґрунтовано необхідність створення компонентно організованої системи, у якій кожний модуль виконує окрему функцію.

У процесі проєктування було визначено структуру програмного продукту, основні модулі та логіку їхньої взаємодії. Програма побудована за компонентно-модульним принципом, що відповідає особливостям Unity. Було спроектовано модуль керування персонажем, модуль взаємодії з предметами, структуру інвентарних даних та користувацький інтерфейс. Такий підхід дав змогу розділити відповідальність між частинами системи та забезпечити зрозумілу архітектуру програмного продукту.

Під час реалізації було використано C#-скрипти, компонентну модель Unity, обробку введення користувача, механізми взаємодії з об'єктами сцени та інвентарну логіку. Програмний продукт забезпечує базове переміщення персонажа, зміну напрямку огляду, підбір предметів, додавання їх до інвентаря та відображення відповідних змін в інтерфейсі. Також було враховано необхідність обмеження керування персонажем під час відкриття інвентаря.

Проведене тестування підтвердило працездатність основних функцій програми та відповідність розробленого програмного продукту поставленим вимогам. Було перевірено запуск сцени, переміщення персонажа, поворот камери, стрибок, відкриття інвентаря, підбір предметів та оновлення слотів інвентаря. Результати тестування показали, що основні сценарії виконуються коректно, а програма здатна реагувати на некоректні налаштування без критичного порушення роботи.

Подальше вдосконалення програмного продукту може передбачати розширення інвентарної системи, додавання нових типів предметів, покращення користувацького інтерфейсу, реалізацію збереження даних між сесіями та впровадження автоматизованого тестування.

Вихідний код програмного продукту розміщено у GitHub-репозиторії:

<https://github.com/illiasmirnov/Inomiriyal.git> .